Health Data Bank

Maintenance Manual

Introduction 6
APIs 7
Admin API: 8
Database Viewer: 8
User Management: 9
View-As: 13
Audit Logs: 14
Account Requests and Deletion Requests: 16
Dashboard Stats: 19
Database Backup and Restore: 20
1.1 Configuration 20
1.2 Data Models 20
1.3 Endpoints 21
1.4 Manual Backup — mysqldump Invocation 21
1.5 Restore — Four-Step Process 22
1.6 Security — Path Traversal Protection 23
System Settings: 23
1.1 Data Models 23
1.2 Endpoints 24
1.3 Storage — INSERT … ON DUPLICATE KEY UPDATE 24
3.4 Caching — services/settings.py 25
3.5 Seeded Defaults 26
Assignments API: 27
Assignment Creation: 28
Assignment Target Preview: 30
List Survey Assignments: 30
My Assignments: 31
Update Assignment: 32
Delete Assignment: 32
Auth API: 34
Password Hashing: 34
Account Creation: 35
Account Requests: 35
Login: 36
Profile Completion: 37
Account Deletion Request: 38
Consent API: 39
Consent Versioning: 39
Admin Exemption: 39
Consent Status: 40
Consent Submission: 40

HCP API:..... 42
The Link Model and Status Lifecycle:..... 42
Requesting a Link:..... 42
Listing Links:..... 43
Responding to a Link:..... 44
Consent Revocation and Restoration:..... 44
Removing a Link:..... 45
HCP Patients API:..... 46
Access Control Architecture:..... 46
Listing Patients:..... 47
Listing Patients Surveys:..... 47
Fetching Patient Responses:..... 48
Health Tracking API:..... 49
1.1 Enumerations..... 49
1.2 Pydantic Response Models..... 50
1.3 Entry Submission Models..... 51
1.4 Participant Endpoints..... 52
1.5 Admin Endpoints..... 55
1.6 Researcher Endpoints..... 57
1.7 Database Schema..... 61
1.8 Localization..... 63
Messaging API:..... 64
Permission Model:..... 64
Conversation Creation:..... 65
Listing Conversations:..... 65
Reading and Sending Messages:..... 66
Friend Requests:..... 67
Participant API:..... 69
Draft Save and Load:..... 69
Survey Submission:..... 69
Listing Surveys:..... 71
Responded Surveys with Answers:..... 72
Response Comparison:..... 72
Chart Data:..... 73
Question Bank API:..... 75
Categories:..... 75
Creating a Question:..... 75
Listing Questions:..... 76
Fetching and Updating Individual Questions:..... 76
Deleting a Question:..... 77
Research API:..... 79

Privacy Architecture:.....79
Per-Survey Endpoints:.....80
Cross-Survey Endpoints:.....81
Responses API:.....83
The Submission Endpoint:.....83
The Submission Endpoint:.....84
Sessions API:.....86
This file manages the session lifecycle and provides the primary endpoint the frontend uses to understand the current user's identity and state. It also contains the core session infrastructure, token generation, hashing, and cookie configuration, that other parts of the codebase import and rely on.....86
Session Infrastructure:.....86
Session Creation:.....87
Session Validation:.....87
Logout:.....88
Session Info:.....88
Surveys API:.....90
The Survey Data Model:.....90
Helper Functions:.....90
Creating Surveys:.....91
Listing and Fetching Surveys:.....91
Updating Surveys:.....92
Publishing and Closing Surveys:.....92
Deleting Surveys:.....93
Templates API:.....94
Templates vs Surveys:.....94
Creating and Listing Templates:.....95
Updating Templates:.....95
Deleting Templates:.....95
Duplicating Templates:.....96
TOS API:.....97
Duplicating Templates:.....97
Two Factor Authentication API:.....98
Enrolling 2FA:.....98
Confirming Enrollment:.....99
Verifying a Login Challenge:.....99
Disabling 2FA:.....100
Users API:.....102
Data Model and Helpers:.....102
Listing and Fetching Users:.....103
Creating Users:.....103

Updating Users:.... 104
Status Toggle:.... 104
Deleting Users:.... 104
Flutter Widgets....106
Researcher Dashboard Sidebar:.... 107
Researcher Header:.... 109
Researcher Scaffold:....110
Global Widgets:.... 111
AppFilledButton....112
AppLongButton.... 113
AppOutlinedButton....113
AppTextButton....114
AppTabButton....114
Relationship to AppTheme....115
Impact of Changes....116
How to Extend or Modify....116
Audit Log.... 118
How It Works....118
Semantic Action Classification....119
What Gets Written....120
Modifying the Route Map....120
Security Structure....122
Password.... 122
Two Factor Authentication.... 122
Sanitization.... 122
Session Tokens....123
Salt/Hash.... 123
Anonymization.... 123
Backup....123
Rate Limiting....123
Audit Log....124
Recommendations....124

Introduction

This is a maintenance manual written for the purpose of instructing administrators and tech staff on how to maintain and expand the Health Databank application. The largest sections will be the explanation of the numerous APIs this software utilizes. But it will also go over the security structure, explanation of flutter widgets, and the maintenance of the audit log. There will also be a miscellaneous section for other things that affect all users or do not fit in the pre-established categories.

APIs will be explained in a single section and in alphabetical order. Flutter widgets will just be a small familiarization and explanation of how to expand, modify, or create new widgets and what widgets might want to be created in the near future. The audit log will explain how it fetches data and how fetching can be modified. The security section will go into the layered security structure set up for version 1.0 of this piece of software, how it works, how it can be changed, and potential expansions for the security section.

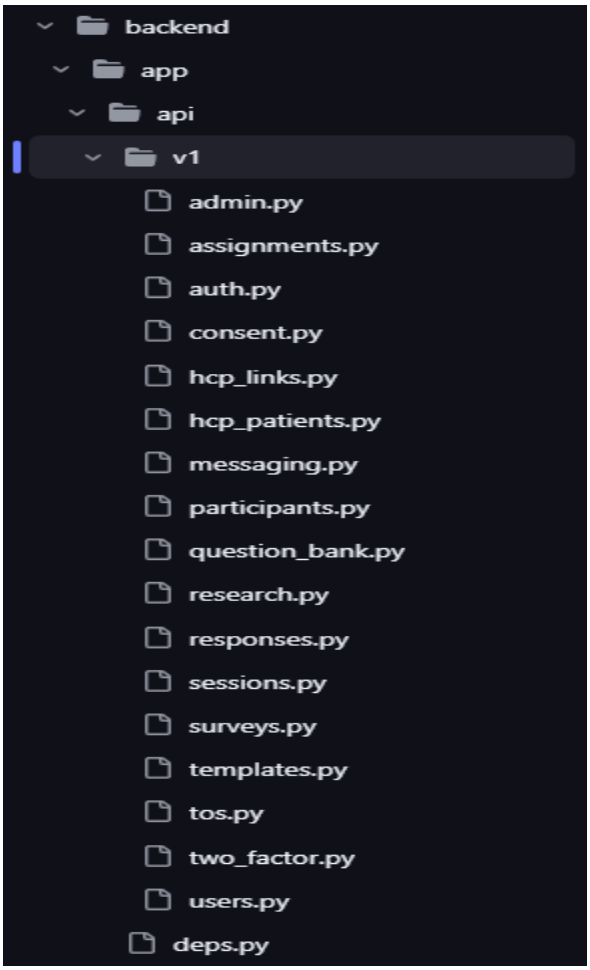
It is recommended that, due to this maintenance manual talking about security and internal workings of the software, that it not be shared outside of trusted members. This could potentially create breaches of security or allow malicious actors to access or destroy data.

It is also recommended that any administrators or tech workers familiarize themselves with the file structure. There will be references to folders or files within this manual, and all pictures on file structure will be a picture from a GitHub repository for clarity; your file structure may not look like that due to a different viewing environment.

There may be some pictures within the manual that look different or contain different material than others, this is due to different versions being used as reference. This can be updated, but the time required to reorient everything was not possible before finalization of the product.

APIs

Upon initial viewing of the API structure, you will see these API files:



This is your entire file structure for APIs within the backend. Each file handles a different service. The names of the files represent what they are handling, and are meant to be descriptive.

In this manual, the titles will be their filename without their extension unless there is specifically a reason to mention it.

Admin API:

The Admin API handles all activities that the Admin user type (role ID 4) is capable of. At the top there is a comment that gives a quick overview of the Endpoints and Security standards.

```
"""
Admin API - Database Viewer & User Management

Endpoints:
- GET    /api/v1/admin/tables                          - List all database tables
- GET    /api/v1/admin/tables/{name}                   - Get table schema and data
- GET    /api/v1/admin/tables/{name}/data              - Get table data only
- POST   /api/v1/admin/users                           - Create user with temp password
- POST   /api/v1/admin/users/{user_id}/reset-password  - Reset user password (admin only)
- POST   /api/v1/admin/users/{user_id}/send-reset-email - Send password reset email
- POST   /api/v1/admin/users/{user_id}/view-as         - Start viewing as user (system admin only)
- POST   /api/v1/admin/view-as/end                     - End view-as mode
- POST   /api/v1/admin/users/{user_id}/impersonate     - [DEPRECATED] Use view-as instead
- POST   /api/v1/admin/impersonate/end                 - [DEPRECATED] Use view-as/end instead

SECURITY:
- All queries use parameterized statements to prevent SQL injection.
- Sensitive data (passwords, tokens) is excluded from responses.
- Temporary passwords are generated using cryptographically secure methods.
- Password reset requires admin authentication.
- View-as/impersonation requires system administrator role (RoleID = 4).
"""
```

We'll begin to go down the list of endpoint groups, specifically what they handle and how it can be modified.

Database Viewer:

The database viewer (a UI element within the application for the administrator role) provides the administrators the ability to inspect the database live through this API. THe following endpoints are used for this task:

```
- GET    /api/v1/admin/tables                          - List all database tables
- GET    /api/v1/admin/tables/{name}                   - Get table schema and data
- GET    /api/v1/admin/tables/{name}/data              - Get table data only
```

The first endpoint, GET /tables, returns a list of every base table in the current database along with schema metadata for each one. This list is discovered dynamically via the Information_Schema function found below, meaning this API function does not need to be changed if new tables are added.

The second endpoint, GET /tables/{name}, returns the schema nad a page of data for a single named table. And the third, GET /tables/{name}/data, returns only the data with no schema metadata. Column metadata includes the column's name, SQL type, whether it's a primary key, whether it's a foreign key and what it references, and whether it's nullable. This is all derived from the functions Information_Schema.columns and Information_Schema.Key_column_usage.

```
SENSITIVE_COLUMNS = {
    'Auth': ['PasswordHash'],
    'Sessions': ['TokenHash'],
    'mfa_challenges': ['TokenHash'],
}
```

The Sensitive_Columns dictionary is used to define all tables which are not to appear in any response. They're excluded by the get_table_schema() function and the Select statement built by get_table_data(). This means if someone calls the data endpoints directly, they will not receive a password hash or session token in response. If the schema ever gains new sensitive columns, this must be updated manually or they will be exposed through this viewer.

User Management:

User management covers all interactions that an administrator can do towards a user account. This section covers the following endpoints:

```
- POST   /api/v1/admin/users                          - Create user with temp password
- POST   /api/v1/admin/users/{user_id}/reset-password  - Reset user password (admin only)
- POST   /api/v1/admin/users/{user_id}/send-reset-email - Send password reset email
```

Administrators are able to create accounts. This section is designed for when, instead of utilizing the normal self-registration, the administrator must forcibly create a user account.

```python
# ==============================================================================
# Admin User Creation Endpoint
# ==============================================================================

class AdminUserCreate(BaseModel):
    """Request body for admin-initiated user creation."""
    email: EmailStr = Field(..., description="User's email address")
    first_name: str = Field(..., min_length=1, max_length=50, description="User's first name")
    last_name: str = Field(..., min_length=1, max_length=50, description="User's last name")
    role_id: int = Field(default=1, ge=1, le=4, description="Role ID (1=Participant, 2=Researcher, 3=Admin, 4=SysAdmin)")

    @field_validator('email', 'first_name', 'last_name', mode='before')
    @classmethod
    def sanitize(cls, v):
        return sanitized_string(v)

    @field_validator('email')
    @classmethod
    def normalize_email(cls, v: str) -> str:
        return v.lower().strip()

    @field_validator('first_name', 'last_name')
    @classmethod
    def strip_whitespace(cls, v: str) -> str:
        return v.strip()
```

The admin will supply an email, first name, last name, and roleID (this dictates what type of account is created). The API will generate a temporary password with the generate_temp_password() function, which uses Python's secrets module to produce a 16 character password containing at least one uppercase letter, one lowercase letter, one digit, and one special character.

The positions of these guaranteed characters are then shuffled using secrets.SystemRandom() so the special character isn't always in a predictable position. The password is hashed with hash_password() before being written to the Auth table of the database. The MustChangePassword flag is then set to True for the account, which will force the user to change it on first login. Afterwards an email containing the temporary password is then sent.

```python
# ============================================================================
# Password Reset Endpoints
# ============================================================================

class PasswordResetRequest(BaseModel):
    """Request body for password reset."""
    new_password: str = Field(..., min_length=8, description="New password (minimum 8 characters)")

    @field_validator('new_password')
    @classmethod
    def validate_password(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError('Password must be at least 8 characters long')
        return v


class PasswordResetResponse(BaseModel):
    """Response for successful password reset."""
    message: str
    user_id: int
```

The POST /users/{user_id}/reset-password endpoint lets an admin set a new password for any user directly. The caller will supply the new password in the request body, which must be longer than 8 characters. The API looks up the user's AuthID via their AccountID, hashes the supplied password, and updates the Auth table. There is no MustChangePassword flag set; but could be added if required.

```python
@router.post("/users/{user_id}/send-reset-email")
async def send_password_reset_email(
    user_id: int,
    request: SendResetEmailRequest,
) -> SendResetEmailResponse:
    """
```

POST /users/{user_id]/send-reset-email is a companion to the reset endpoint above. It send an email to the user containing a temporary password, but it does not generate or set the password itself; the caller must supply the temporary password in the request body.

```python
@router.delete("/users/{user_id}/purge", status_code=status.HTTP_200_OK)
async def purge_user_data(
    user_id: int,
    request: Request,
    scrub_audit_metadata: bool = True,
```

There is also a purge data endpoint. It permanently removes a user and their associated data if required. The deletion sequence is ordered to respect foreign key constraints: sessions are deleted first, then survey responses, survey assignments, 2FA records, then the AccountData row within the database, and then finally the Auth row containing the password hash. If any step fails the transaction is rolled back and nothing is deleted.

An optional query parameter scrub_audit_metadata controls what happens to audit log entries that reference this user. Rather than deleting those rows the endpoint nulls the IpAddress, UserAgent, and MetadataJSON fields on any AuditEvent where ActorAccountID matches the purged user. This preserves that an event occurred while removing the identifying information.

View-As:

```
- POST   /api/v1/admin/users/{user_id}/view-as         - Start viewing as user (system admin only)
- POST   /api/v1/admin/view-as/end                     - End view-as mode
- POST   /api/v1/admin/users/{user_id}/impersonate     - [DEPRECATED] Use view-as instead
- POST   /api/v1/admin/impersonate/end                 - [DEPRECATED] Use view-as/end instead
```

There are two systems of impersonation within this codebase. The legacy system uses the /users/{user-id}/impersonate and /impersonate/end endpoints. This legacy system generates a completely new session token for the target user's account, with the admin's AccountID stored in the ImpersonatedBy column.

The current system used utilizes the /users/{user_id}/view-as and /view-as/end endpoints. The difference is that rather than creating a new session, it mutates the admin's existing session by setting a ViewingAsUserID column. The admin's token never changes which eliminates all token-juggling on the frontend. From the admin's perspective they're still in the same session, the server just interprets their request as if they were the target user while ViewingAsUserID is set.

The system enforces the same pre-conditions before allowing it to proceed. The caller must have a valid, non-expired session. Their role must be exactly 4 (administrator). And the target user must exist and be active. Self-targeting the impersonation call will result in a 400 error, specifically being rejected.

View-as endpoints also write to the AuditEvent table. Starting view-as writes a view_as_start event recording the admin's account ID and the target user's ID. Ending it writes a view_as_end event.

Audit Logs:

The audit log section, to the administrator, is designed for viewing the audit log:

```python
# ==========================================
# Audit Log Endpoints
# ==========================================

class AuditEventResponse(BaseModel):
    """Single audit event in response."""
    audit_event_id: int
    created_at: str
    request_id: Optional[str] = None
    actor_type: str
    actor_account_id: Optional[int] = None
    actor_email: Optional[str] = None
    actor_name: Optional[str] = None
    ip_address: Optional[str] = None
    user_agent: Optional[str] = None
    http_method: Optional[str] = None
    path: Optional[str] = None
    action: str
    resource_type: str
    resource_id: Optional[str] = None
    status: str
    http_status_code: Optional[int] = None
    error_code: Optional[str] = None
    metadata: Optional[Dict[str, Any]] = None


class AuditLogResponse(BaseModel):
    """Response for audit log listing."""
    events: List[AuditEventResponse]
    total: int
    limit: int
    offset: int


@router.get("/audit-logs")
```

The audit log endpoints take the AuditEvent table, which serves as the security log for the platform. The main endpoint, GET /audit-logs, returns a paginated list of events with a set of filters.

Filtering can be applied on any combination of the action field, the status field, a specific actor_account_id to see everything a particular user did, a resource_type to scope results to a particular entry type, the http_method of the original request, a free-text search that matches against both the path and action fields using SQL LIKE, and date range filters.

All filters are combined with ‘&’, and all values are passed as parameterized query arguments to prevent injection. Pagination uses limit and offset, set internally.

Actor information like name and email are joined from AccountData using a Left Join, so events where the actor account has since been deleted will still appear but with null actor fields. IP addresses are stored as binary in the database and decoded to human-readable notation in the response handler with separate handling for IPv4 and IPv6. The MetadataJSON field, if present, is parsed from a JSON string into a native object before being included in the response.

The two helper endpoints, GET /audit-logs/actions and GET /audit-logs/resource-types, return the values currently present in the AuditEvent table for those fields. These are intended to populate filter dropdown menus in the admin UI and are dynamically generated from live data, so they will automatically reflect any new event types that get added to the system.

Account Requests and Deletion Requests:

These two sections manage the approval workflows for users joining and leaving the platform:

```python
# ============================================================
# Account Request Management Endpoints
# ============================================================

class AccountRequestResponse(BaseModel):
    """Response model for an account request."""
    request_id: int
    first_name: str
    last_name: str
    email: str
    role_id: int
    role_name: Optional[str] = None
    birthdate: Optional[str] = None
    gender: Optional[str] = None
    gender_other: Optional[str] = None
    status: str
    admin_notes: Optional[str] = None
    reviewed_by: Optional[int] = None
    created_at: str
    reviewed_at: Optional[str] = None


class AccountRequestCountResponse(BaseModel):
    """Response for pending account request count."""
    count: int


class AccountRequestRejectBody(BaseModel):
    """Request body for rejecting an account request."""
    admin_notes: Optional[str] = Field(None, max_length=1000)

    @field_validator('admin_notes', mode='before')
    @classmethod
    def sanitize(cls, v):
        return sanitized_string(v)


@router.get("/account-requests")
```

Account requests represent self-registration submissions waiting for review.

The list endpoint of GET /account-requests defaults to showing only pending requests but accepts a status query parameter to retrieve approved or rejected ones as well.

The count endpoint returns only the number of pending requests, intended to drive a notification badge in the admin UI without fetching full records.

Approving a request, POST /account-requests/{id}/approve, follows the same credential generation flow as direct user creation. A temporary password is generated with generate_temp_password(), hashed, then stored with MustChangePassword being changed to True. An Auth record is created first, then an AccountData record referencing it. The request row is then updated to Status ‘approved’ and ReviewedAt is stamped with the current UTC time. A welcome email is sent. The endpoint also guards against approving a request for an email that has already been registered, returning a 409 conflict in that case.

```
@router.post("/account-requests/{request_id}/reject")
async def reject_account_request(
    request_id: int,
    body: Optional[AccountRequestRejectBody] = None,
) -> dict:
    """Reject an account request with optional admin notes."""
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    try:
        cursor.execute(
            "SELECT RequestID, FirstName, Email, Status FROM A
            (request_id,),
        )
        req = cursor.fetchone()
```

Rejecting requests is simpler. It just updates the status to rejected, optionally records admin notes, and sends a rejection email to the applicant. With no account being created. The route is POST /acount-requests/{request_id}/reject.

Deletion requests are the opposite, users who have asked for their accounts to be deleted. The listing and count endpoints work identically to their account request counterparts.

```python
# ================================================
# Account Deletion Request Endpoints
# ================================================

class DeletionRequestResponse(BaseModel):
    """A user-submitted account deletion request."
    request_id: int
    account_id: int
    first_name: str
    last_name: str
    email: str
    status: str
    admin_notes: Optional[str] = None
    reviewed_by: Optional[int] = None
    requested_at: str
    reviewed_at: Optional[str] = None


class DeletionRequestRejectBody(BaseModel):
    admin_notes: Optional[str] = Field(None, max_l

    @field_validator('admin_notes', mode='before')
    @classmethod
    def sanitize(cls, v):
        return sanitized_string(v)


@router.get("/deletion-requests", dependencies=[De
```

Approving a deletion is more thorough than the purge endpoint in the user management section. It performs the same Foreign Key ordered cleanup but goes further. Survey responses are anonymized rather than deleted by setting ParticipantID to null, consent records are removed, messages sent by the user are deleted, and creator references within the survey table are set to null rather than leaving foreign keys.

The deletion request row itself is marked ‘approved’ with the reviewing admin’s ID before the AccountData now is removed. This ordering matters because once AccountData is gone, the join needed to display the request would fail. Any failure in the process results in rollback.

Rejecting a deletion, in POST /deletion-requests/{id}/reject, means the user has changed their mind or the request was invalid. The account is reactivated by setting IsActive to ‘True’, and the request is marked ‘rejected’ with the admin’s notes. The user can log in immediately after this.

Dashboard Stats:

THe dashboard stats endpoint, GET /dashboard/stats, aggregates a broad snapshot of the system's state into a single response, designed to populate an admin overview page without the frontend needing to make a dozen separate requests.

```
# ==========================================
# Dashboard Stats
# ==========================================

class DashboardStatsResponse(BaseModel):
    total_users: int
    active_users: int
    users_by_role: Dict[str, int]
    new_users_30d: int
    total_surveys: int
    published_surveys: int
    draft_surveys: int
    closed_surveys: int
    total_responses: int
    pending_account_requests: int
    pending_deletion_requests: int
    recent_account_requests: List[Dict[str,
    recent_deletion_requests: List[Dict[str,
    recent_logins: List[Dict[str, Any]]


@router.get("/dashboard/stats", response_mod
```

The response is assembled from twelve sequential database queries. User statistics include the total account count, the count of active participants, a breakdown of active accounts by role name, and the number of new accounts created in the past 30 days. Survey statistics cover the total survey count and separate counts for each publication status: published, draft, and closed. The total response count across all surveys is also included.

The pending request counts, one for account requests and another for deletion requests, are the same figures that drive the badge counters on their respective list endpoints. Alongside the counts, the response includes the five most recent pending account requests and five most recent pending deletion requests, with enough fields to be displayed in a summary table without further API calls.

Finally, the response includes the ten most recent active sessions, showing who is currently logged in, their role, and when the session was created. These are filtered by IsActive 'true' on the sessions table, so expired or invalidated sessions are excluded.

All twelve queries run sequentially on a single database connection and cursor, so the snapshot is not strictly atomic; a busy system might result in counts being marginally inconsistent with each other. For an admin dashboard, providing data at a glance, this is generally acceptable, but it is worth noting in case you see any minor inconsistencies when operating the system.

Database Backup and Restore:

File: backend/app/api/v1/admin.py (lines 2137–2448) | Router prefix: /api/v1/admin

The backup subsystem provides on-demand mysqldump backups, scheduled backup listing, direct file download, and one-click restore with automatic pre-restore safeguarding. Only administrators (RoleID 4) may trigger backups, delete manual backups, or restore the database. The download endpoint intentionally does not require authentication so administrators can share a direct download link, but the file path is validated against path-traversal attacks.

1.1 Configuration

The backup root directory is controlled by the BACKUPS_DIR environment variable (default /backups). Four sub-directories are maintained automatically.

admin.py — Backup configuration constants

```
_BACKUPS_ROOT = Path(os.getenv('BACKUPS_DIR', '/backups'))

# Sub-directories managed by backup system
_BACKUP_TYPES = ('daily', 'weekly', 'monthly', 'manual')

# Maximum manual backups retained (oldest pruned automatically)
_MANUAL_KEEP = 10

# Scheduled backups are created by cron jobs in the container and are
# never deleted via the API.  Only 'manual' backups can be deleted.
```

Database credentials are sourced from DATABASE_URL (parsed via urlparse) or from individual MYSQL_HOST / MYSQL_USER / MYSQL_PASSWORD / MYSQL_DATABASE environment variables as a fallback. The password is passed through the MYSQL_PWD environment variable to avoid it appearing on the process command line.

1.2 Data Models

admin.py — Backup data models

```
class BackupInfo(BaseModel):
```

```
    filename:     str       # e.g. 'daily/healthdatabase_2026-03-31_020000.sql.gz'
    backup_type:  str       # 'daily' | 'weekly' | 'monthly' | 'manual'
    created_at:   datetime
    size_bytes:   int
    size_human:   str       # e.g. '4.2 MB'

class RestoreResult(BaseModel):
    pre_backup_filename: str   # manual/<db>_pre_restore_<ts>.sql.gz
    pre_backup_size_human: str
    restored_file:   str       # <type>/<filename>
    migrations_run:  int       # count of migration .sql files applied post-restore
    message:         str       # human-readable summary
```

1.3 Endpoints

Method	Path	Auth (RoleID)	Description
GET	/api/v1/admin/backups	public	List all backup files across all types, newest first.
POST	/api/v1/admin/backups/trigger	4	Trigger an immediate manual mysqldump. Prunes oldest if > 10 manual backups exist.
GET	/api/v1/admin/backups/{backup_type}/{filename}/download	public	Stream the .sql.gz file. Filename validated against path traversal.
DELETE	/api/v1/admin/backups/manual/{filename}	4	Delete a manual backup. Daily/weekly/monthly cannot be deleted via API.
POST	/api/v1/admin/backups/{backup_type}/{filename}/restore	4	Full 4-step database restore (see below).

1.4 Manual Backup — mysqldump Invocation

The trigger endpoint runs mysqldump as a subprocess, capturing stdout directly into a gzip stream. The --single-transaction flag ensures a consistent snapshot without

locking tables. The --skip-ssl flag is required for compatibility with the MariaDB client bundled in the container.

admin.py — mysqldump subprocess call (trigger_backup)

```
proc = subprocess.run(
    [
        'mysqldump',
        f'-h{creds["host"]}',
        f'-P{creds["port"]}',
        f'-u{creds["user"]}',
        '--skip-ssl',
        '--databases', creds['database'],
        '--single-transaction',
        '--routines',
        '--triggers',
        '--events',
        '--no-tablespaces',
    ],
    capture_output=True,
    env={**os.environ, 'MYSQL_PWD': creds['password']},
    check=True,
)
with gzip.open(out_path, 'wb') as gz:
    gz.write(proc.stdout)
```

1.5 Restore — Four-Step Process

The restore endpoint follows a strict four-step sequence to minimize risk. If Step 1 (pre-restore backup) fails, the restore is aborted immediately before any changes are made to the live database.

- Step 1 — Input validation: backup_type checked against allowlist; filename checked for '..' , '/', '\\'; must end with .sql.gz; file existence verified.
- Step 2 — Automatic pre-restore backup: a new mysqldump is created in manual/ as <db>_pre_restore_<timestamp>.sql.gz before any restore begins.
- Step 3 — Database restore: the selected .sql.gz is decompressed in memory and piped to the mysql CLI. The SQL file contains CREATE DATABASE / USE statements so the target database is fully replaced.
- Step 4 — Migration replay: all .sql files in /code/app/migrations/ are applied in sorted order to bring the schema to the latest version. Already-applied migrations are safe to re-run (they use CREATE TABLE IF NOT EXISTS / INSERT IGNORE patterns).

admin.py — Restore steps 2-4 (restore_backup)

```
# Step 2 — pre-restore backup
subprocess.run(['mysqldump', ...], capture_output=True, check=True)
with gzip.open(pre_path, 'wb') as gz: gz.write(proc.stdout)

# Step 3 — restore
with gzip.open(file_path, 'rb') as gz_in:
```

```
        sql_bytes = gz_in.read()
    subprocess.run(['mysql', '-h...', '-u...', '--skip-ssl'],
        input=sql_bytes, capture_output=True, check=True)

    # Step 4 — run migrations
    for mfile in sorted(Path('/code/app/migrations').glob('*.sql')):
        subprocess.run(['mysql', '-h...', creds['database']],
            input=mfile.read_bytes(), capture_output=True)
        migrations_run += 1
```

1.6 Security — Path Traversal Protection

Every endpoint that accepts a filename parameter applies three guards before constructing any filesystem path. This prevents directory traversal attacks that could expose or overwrite files outside the backups root.

admin.py — Path traversal guards

```
    # Applied in download, delete, and restore endpoints
    if backup_type not in _BACKUP_TYPES:
        raise HTTPException(400, 'Invalid backup type.')

    if '..' in filename or '/' in filename or '\\' in filename:
        raise HTTPException(400, 'Invalid filename.')

    if not filename.endswith('.sql.gz'):
        raise HTTPException(400, 'Only .sql.gz files are allowed.')

    file_path = _BACKUPS_ROOT / backup_type / filename  # safe after validation
```

System Settings:

File: backend/app/api/v1/admin.py (lines 2450–2551) | Router prefix: /api/v1/admin

System Settings provides a single-table configuration store for runtime-adjustable system behaviour. Settings are read by multiple other API modules (k-anonymity threshold by the Health Tracking researcher endpoints, maintenance_mode by the middleware banner, registration_open by the auth module, etc.). A 30-second in-memory cache in services/settings.py prevents excessive database reads on high-traffic pages.

1.1 Data Models

admin.py — System Settings models

```
    class SystemSettingsResponse(BaseModel):
        k_anonymity_threshold:    int    # min distinct respondents for research data
        registration_open:        bool   # allow new participant sign-ups
        maintenance_mode:         bool   # show maintenance banner on all pages
        maintenance_message:      str    # banner body text (max 500 chars)
        maintenance_completion:   str    # estimated completion time (max 200 chars)
```

```
    max_login_attempts:       int    # 0 = unlimited
    lockout_duration_minutes: int    # minutes before failed-login lockout clears
    consent_required:         bool   # require participants to accept consent form
    defaults:                 dict   # seeded default values for 'Reset' UI labels

class SystemSettingsUpdate(BaseModel):
    k_anonymity_threshold:    int  = Field(..., ge=1)
    registration_open:        bool
    maintenance_mode:         bool
    maintenance_message:      str  = Field('', max_length=500)
    maintenance_completion:   str  = Field('', max_length=200)
    max_login_attempts:       int  = Field(..., ge=0)  # 0 = unlimited
    lockout_duration_minutes: int  = Field(..., ge=1)
    consent_required:         bool
```

1.2 Endpoints

Method	Path	Auth (RoleID)	Description
GET	/api/v1/admin/settings	public (no auth)	Return current settings. Available to all roles so frontend can show maintenance banner and check registration status before login.
PUT	/api/v1/admin/settings	4	Atomically update all settings. Each key written via INSERT...ON DUPLICATE KEY UPDATE. Cache invalidated after commit.

1.3 Storage — INSERT … ON DUPLICATE KEY UPDATE

The SystemSettings table uses a key/value schema. All settings are upserted in a loop so the same code path handles both first-time creation and subsequent updates. The entire loop is committed atomically — a failure rolls back all changes to prevent partial updates.

admin.py — Settings upsert loop (update_settings)

```
# PUT /settings — write all keys atomically
conn = get_db_connection()
cur  = conn.cursor()
try:
```

```
    for key, value in updates.items():
        cur.execute(
            '''
            INSERT INTO SystemSettings (SettingKey, SettingValue, UpdatedBy)
            VALUES (%s, %s, %s)
            ON DUPLICATE KEY UPDATE
                SettingValue = VALUES(SettingValue),
                UpdatedBy    = VALUES(UpdatedBy)
            ''',
            (key, value, account['account_id']),
        )
    conn.commit()
    invalidate_cache()  # forces next read to hit DB
except mysql.connector.Error as err:
    conn.rollback()
    raise HTTPException(500, detail=f'Database error: {err}')
```

3.4 Caching — services/settings.py

Settings are cached in-process to avoid a database round-trip on every API request. The cache TTL is 30 seconds. Admin PUT /settings calls invalidate_cache() immediately after commit so the next read reflects the new values with at most one request latency.

services/settings.py — cache pattern

```
# services/settings.py — simplified cache pattern
_cache: dict = {}
_cache_time: float = 0.0
_CACHE_TTL = 30  # seconds

def _load_settings() -> dict:
    global _cache, _cache_time
    if time.time() - _cache_time < _CACHE_TTL:
        return _cache
    conn = get_db_connection()
    cur  = conn.cursor(dictionary=True)
    cur.execute('SELECT SettingKey, SettingValue FROM SystemSettings')
    _cache = {r['SettingKey']: r['SettingValue'] for r in cur.fetchall()}
    _cache_time = time.time()
    return _cache

def invalidate_cache():
    global _cache_time
    _cache_time = 0.0   # forces reload on next call

def get_int_setting(key: str, default: int = 0) -> int:
    return int(_load_settings().get(key, default))

def get_bool_setting(key: str, default: bool = False) -> bool:
    return _load_settings().get(key, str(default)).lower() == 'true'
```

3.5 Seeded Defaults

Default values are seeded in the database migration and mirrored in the _SETTING_DEFAULTS dict in admin.py so the frontend can display 'Reset to default' hints. The defaults dict is included in every SystemSettingsResponse.

admin.py — _SETTING_DEFAULTS (hardcoded fallbacks)

```
_SETTING_DEFAULTS = {
    'k_anonymity_threshold':    '5',
    'registration_open':        'true',
    'maintenance_mode':         'false',
    'maintenance_message':      '',
    'maintenance_completion':   '',
    'max_login_attempts':       '5',
    'lockout_duration_minutes': '30',
    'consent_required':         'true',
}
```

Assignments API:

This API defines two routers. The survey_router handles assignment operations scoped to a specific survey (assigning, listing previewing) and requires RoleID 2 (researcher) or 4 (administrator). The main router handles assignment operations from the perspective of an individual assignment record or the current user, and is accessible to RoleID's 1,2, and 4; meaning participants can read their own assignments, while researchers and admins can manage them.

```python
# Created with the Assistance of Claude Code
# backend/app/api/v1/assignments.py
"""
Survey Assignment API

Endpoints:
- POST   /api/v1/surveys/{id}/assign      - Assign survey to user(s)
- GET    /api/v1/surveys/{id}/assignments - List assignments for a survey
- GET    /api/v1/assignments/me           - Get my assignments (current user)
- PUT    /api/v1/assignments/{id}         - Update assignment (due date, status)
- DELETE /api/v1/assignments/{id}         - Remove assignment

SECURITY: All queries use parameterized statements to prevent SQL injection.
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from pydantic import BaseModel
from typing import Optional, List, Union
from enum import Enum
from datetime import datetime
from ...utils.utils import get_db_connection
from ..deps import require_role

router = APIRouter(dependencies=[Depends(require_role(1, 2, 4))])
```

Assignment Creation:

This uses the POST /surveys/{survey_id}/assign endpoint, and it handles two different workflows under a single route, determined by the content of the body.

```python
# Pydantic Models
class AssignmentCreate(BaseModel):
    account_id: Optional[int] = None
    account_ids: Optional[List[int]] = None
    assign_all: bool = False
    gender: Optional[str] = None
    age_min: Optional[int] = None
    age_max: Optional[int] = None
    due_date: Optional[datetime] = None
```

Before each path runs, the endpoint validates that the target survey exists and that PublicationStatus is ‘published’. Attempting to assign a draft or closed survey returns a 400 error with the current status included in the message, which can be used for diagnosing misconfiguration. This guard prevents participants from receiving assignments for surveys that aren’t ready.

Bulk assignment when any of assign_all, gender, age_min, or age_max are set in the request body. In this mode the endpoint ignores any account_id or account_ids fields and instead queries AccountData for all active participants that match the supplied demographic filters. Gender is matched exactly. Age is computed at query time, at a year-level approximation.

For each participant in the targeted set, the bulk path checks for a pre-existing assignment and skips the user if one is found, rather than raising an error. This makes bulk assignment idempotent, re-running the same assignment is safe and will simply report the already-assigned users in the ‘skipped’ count of the BulkAssignmentResult response. The response tells the caller how many users were targeted, how many were newly assigned, and how many were skipped, which is useful for auditing the operation.

Individual assignment is triggered when assign_all is false and no demographic filters are set. The caller must provide either an account_id or account_ids; omitting

both returns a 422 error. Unlike bulk mode, individual mode is not idempotent; any user in the list that already has an assignment for this survey, the endpoint will raise a 409 conflict immediately. This means that if you supply a list of five account IDs and the third already has the assignment, the first two will have been inserted and committed before the conflict is detected.

The response type varies depending on how many accounts were targeted. A single AssignmentResponse object is returned when exactly one account was assigned, and a list is returned for multiple. The polymorphic return type means the caller must be prepared to handle the three possible return types.

Assignment Target Preview:

This endpoint allows a researcher to see how many participants would be affected by a bulk assignment before committing to it.

```python
@survey_router.get("/{survey_id}/assignments/preview-target")
async def preview_assignment_target(
    survey_id: int,
    gender: Optional[str] = Query(None),
    age_min: Optional[int] = Query(None),
    age_max: Optional[int] = Query(None),
) -> AssignmentTargetPreview:
    """Preview how many participants would be newly assigned."""
    conn = get_db_connection()
    cursor = conn.cursor()
```

It accepts the same gender, age_min, and age_max filters as the bulk assignment path and returns three numbers: the total number of participants matching the demographic filters (total_targeted), how many of those are already assigned to this survey (already_assigned), and how many would be newly assigned (assignable).

The already assigned count is computed using an IN clause with a dynamically built placeholder list, one %s per targeted participant ID. This is safe from injection because the IDs come from a prior database query (not from user input), but could be a performance concern if the participant pool grows very large. For most deployments this is unlikely to matter, but is worth considering for scaling purposes.

List Survey Assignments:

This endpoint returns all SurveyAssignment rows for a given survey, optionally filtered by ‘status’. Results are ordered newest first by AssignedAt. The survey existence check here uses a bare SELECT ‘SurveyID’ rather than checking publication status, so assignments can be listed for surveys in any state, including drafts and closed surveys.

My Assignments:

```
# Assignment-specific endpoints
@router.get("/me")
async def get_my_assignments(use
    """Get all assignments for t
    account_id = user["effective_
    conn = get_db_connection()
    cursor = conn.cursor()

    try:
        query = """
            SELECT
```

This endpoint, GET /assignments/me, is designed for participants to retrieve their own assignment list. It derives the caller's identity from 'user["effective_account_id"]', which means it respects the view-as system from the admin API; if a system administrator is currently viewing as a participant, this endpoint will return that participant's assignments rather than the admin's.

The response model, MyAssignmentResponse, differs from the researcher-facing model in that it includes the survey title rather than exposing the raw account_id, which is the appropriate level of detail for a participant-facing view.

The due date is computed with COALESCE(sa.DueDate, s.EndDate); if no specific due date was set on the assignment itself, the survey's overall end date is used as a fallback. This means participants always see a meaningful deadline if one exists at either level.

An optional status query parameter lets participants filter their list to a specific assignment state, which is useful for building views like "my pending surveys" or "my completed surveys."

Update Assignment:

This endpoint allows updating the due_date and/or status of an existing assignment. Both fields are optional; if neither is supplied in the request body, the handler detects that the updates list is empty, skips the UPDATE query entirely, and immediately re-fetches and returns the unchanged record. This is valid but slightly unusual; a 422 response for a no-op update might be more informative, though the current behaviour is harmless.

When status is set to completed, the handler automatically sets CompletedAt to the current server time (datetime.now()). Note that this uses the local server clock rather than datetime.utcnow() or a database-side UTC_TIMESTAMP(), which could produce inconsistent timestamps if the server's timezone is not UTC. This is worth standardising if the platform operates across timezones.

The status field is constrained by the AssignmentStatus enum, which accepts only pending, completed, and expired. Attempting to set any other value will be rejected by Pydantic before the handler runs.

Delete Assignment:

Assignments can be deleted as long as they are not in a completed state. Attempting to delete a completed assignment returns a 400 error. This guard exists to preserve the integrity of historical completion data; once a participant has submitted responses for a survey, the assignment record that links them should not be removable. Pending and expired assignments can be freely deleted, and the endpoint returns HTTP 204 (No Content) on success, with no response body.

One thing to note: deleting an assignment does not cascade to or remove any Responses rows the participant may have submitted. If a participant partially completed a survey and their assignment is then deleted before they finish, their partial responses would remain in the Responses table with no assignment record linking back to them.

This orphaned data scenario is worth considering if assignment deletion is used as part of any data cleanup workflow.

Auth API:

This file handles everything related to user identity. Account creation, login, password management, MFA challenges, profile completion, and account deletion requests. It is the most security-critical file in the codebase, nearly every other part of the system depends on the session tokens and password hashes it produces.

Password Hashing:

The two utility functions, hash_password() and verify_password(), underpin all credential storage in the system, so it's worth understanding them in detail before looking at the endpoints that use them.

hash_password() uses PBKDF2-HMAC-SHA256, a well-established key derivation function designed specifically for password storage. It generates a 32-byte cryptographic salt using os.urandom(), which reads from the operating system's secure random source. The password is then derived using 210,000 iterations; this iteration count is in line with current OWASP recommendations and makes brute-force attacks significantly more expensive. The output is a single self-describing string in the format pbkdf2_sha256$210000$32$<salt_b64>$<hash_b64>, where the salt and derived key are both base64-encoded. Storing all parameters in the hash string means the verification function never needs external configuration; it can reconstruct the derivation from the stored value alone, which also makes future algorithm migrations straightforward.

verify_password() reconstructs the derived key from the stored string by parsing out the algorithm parameters, re-running the PBKDF2 derivation with the same salt and iteration count, and comparing the result against the stored hash. Critically, the comparison uses hmac.compare_digest() rather than ==. This is important because == in Python short-circuits on the first mismatched byte, which creates a timing side-channel that a sufficiently precise attacker could exploit to infer information about the stored hash. hmac.compare_digest() runs in constant time regardless of where the

mismatch occurs. If parsing or decoding fails for any reason, verify_password() returns False rather than raising an exception, which is the safe default.

Account Creation:

The POST /create_account endpoint handles direct self-registration, where a user supplies their name, email, and chosen password. Input is validated by the AccountCreate Pydantic model before the handler runs: names are sanitized and must be between 1 and 100 characters, email is validated as a proper email address, and the password must be at least 8 characters. The creation sequence mirrors the admin-initiated path: an Auth row is inserted first (containing the password hash), and then an AccountData row is inserted referencing the new AuthID. This ordering is necessary because AccountData.AuthID is a foreign key.

Duplicate email addresses are detected by catching a MySQL IntegrityError with errno 1062 (the duplicate key error code) and returning a 409 Conflict. Other IntegrityError variants are re-raised rather than swallowed, preserving visibility into unexpected constraint violations. Note that unlike the admin-created path, there is no MustChangePassword flag set here; the user chose their own password, so no forced change is needed.

One thing to note about this endpoint relative to the rest of the file: it does not have a rate limit applied. The request_account endpoint below does. If direct self-registration is still an active path in production, adding a rate limit here would be prudent to prevent account enumeration or abuse.

Account Requests:

The POST /request_account is the public-facing registration path, where a user submits a request that must be reviewed by an admin before an account is created. It is rate-limited to 5 requests per 60 seconds per IP address. The endpoint performs two duplicate checks before inserting: it confirms the email is not already in AccountData (i.e. no active account exists), and that it is not already in AccountRequest with a pending status. This prevents both double-submissions and attempts to bypass a

rejection by re-submitting the same email. The role_id field is constrained to 1–3 by Pydantic, meaning requesters cannot self-assign the System Administrator role. Gender is validated against a fixed set of allowed values (Male, Female, Non-Binary, Prefer Not to Say, Other), and an optional gender_other free-text field is available for the Other case.

Login:

The POST /login endpoint is rate-limited to 10 attempts per 60 seconds per IP and handles two meaningfully different outcomes depending on whether the account has MFA enabled.

The login query joins Auth and AccountData on the email address to retrieve the password hash, MustChangePassword flag, role, birthdate, gender, and IsActive status in a single round trip. If no account is found for the email, or if the password verification fails, the same generic error message is returned in both cases, "Invalid email or password", to prevent account enumeration (an attacker cannot distinguish between a wrong email and a wrong password). If the account exists but is inactive, a distinct 403 error with the code "account_deactivated" is returned rather than the generic auth error, which allows the frontend to display a specific message to the user.

If MFA is disabled, the flow proceeds directly to session creation: create_session_for_account() is called with the account ID, IP address, and user agent, and the resulting session token is set as an HttpOnly cookie via set_session_cookie(). The token is explicitly removed from the response body before returning; the cookie is the sole mechanism for carrying the session, and exposing the token in the JSON response would undermine that. The response also carries two convenience flags: must_change_password (so the frontend can redirect to a forced password change screen) and needs_profile_completion (true for participants who haven't yet set their birthdate and gender).

If MFA is enabled, the endpoint does not create a session at all. Instead it generates a short-lived MFA challenge: a 32-byte random token (_new_challenge_token()) is generated, SHA-256 hashed, and stored in the

mfa_challenges table with a 10-minute TTL and a purpose of 'login'. The raw (unhashed) token is returned to the client in the response body as challenge_token. The client must then present this token alongside their MFA code to a separate endpoint (defined elsewhere) to complete authentication and receive a session cookie. This two-step design means an attacker who captures the login response but not the MFA device still cannot authenticate.

LastLogin on the account is updated regardless of whether MFA is enabled, stamping the timestamp at the point of credential verification rather than at the point of full session establishment.

There is a development shortcut in the Login model that warrants explicit attention: a field validator on email checks whether the input contains an @ character, and if not, appends @hb.com automatically. This means submitting "admin" as the email is equivalent to submitting "admin@hb.com". The comment in the source marks this for removal before production, but it must be actively removed; it will not disable itself.

Profile Completion:

This is the POST /complete_profile endpoint. Participants (RoleID = 1) who register without providing a birthdate and gender are flagged via the needs_profile_completion field in the login response. This endpoint allows them to supply those details after their first login. It checks that the caller is actually a participant before updating; researchers and admins cannot use this endpoint. The gender field is sanitized before storage but is not validated against the fixed enum used in AccountRequestCreate, meaning a wider range of gender values can be stored through this path than through the registration request path. If consistency matters, the same VALID_GENDERS validation should be applied here.

Account Deletion Request:

The POST /me/deletion-request endpoint handles self-service deletion requests from authenticated users. It does three things in a single transaction: it sets IsActive = FALSE on the account, revokes all active sessions by marking them inactive with a RevokedAt timestamp, and inserts a pending record into AccountDeletionRequest. After committing, the current session cookie is cleared, logging the user out immediately.

The key design decision here is that no data is actually deleted at this point. The account is deactivated and the user loses access, but all their data remains intact until an administrator reviews and approves the request via the admin API. This gives the platform a window to reject deletion requests if needed (for example, if a participant has outstanding obligations) and keeps the deletion auditable. The actual data cleanup sequence, which differs from the admin-initiated purge, is described in the Admin API documentation.

Consent API:

This API manages the informed consent flow that participants must complete before using the platform. The design is deliberately simple, two endpoints, one to check status and one to submit, but several details are worth understanding for maintenance.

Consent Versioning:

The current consent version is read from the environment variable CURRENT_CONSENT_VERSION at module load time, defaulting to "1.0" if the variable is not set. This is the mechanism by which a consent version upgrade is triggered across the platform: updating the environment variable and redeploying will immediately cause all existing users to show needs_consent: true on the status endpoint, prompting them to re-sign before they can continue. The version string is stored alongside every consent record and on the AccountData row, so the system always knows which version a user signed and whether it matches the current one.

Maintenance note**:** If a new consent document is deployed, the environment variable must be updated. There is no migration step or admin action required; the version check is entirely a string comparison between what is stored on the account and what the environment variable currently holds. Ensure the new version string is distinct from all previous ones.

Admin Exemption:

System Administrators (RoleID = 4) are explicitly exempt from consent requirements throughout this file. The status endpoint returns a hardcoded "signed" response for admins without touching the database at all, and the submit endpoint actively rejects admin submissions with a 400 error. This means admins can use the platform without ever going through the consent flow, which is appropriate since consent is a participant-facing ethical obligation rather than a platform access control.

Consent Status:

For non-admin users, this endpoint queries the ConsentSignedAt and ConsentVersion columns on the AccountData row and compares the stored version against CURRENT_CONSENT_VERSION. The response includes five fields: whether the user has signed (has_signed_consent), which version they signed (consent_version), when they signed it (consent_signed_at as an ISO string), the current required version (current_version), and a convenience boolean (needs_consent) that is simply the inverse of has_signed_consent. The frontend should use needs_consent to gate access and redirect unsigned users to the consent form.

Note that needs_consent is true in two distinct situations: the user has never signed at all (in which case consent_version will be null), and the user has signed a previous version (in which case consent_version will be a non-null string that doesn't match the current one). The frontend may want to handle these cases differently, showing a first-time consent form versus a "our consent document has been updated, please re-sign" message, and can distinguish them by checking whether consent_version is null.

Consent Submission:

The submission endpoint accepts three fields. document_text is the full text of the consent document the user is agreeing to, which must be at least 10 characters. document_language is constrained by a Pydantic regex pattern to exactly "en" or "fr", rejecting any other value before the handler runs. signature_name is the user's typed name serving as their electronic signature, sanitized and limited to 128 characters.

The inclusion of document_text in the request body is a notable design choice. Rather than just recording that a user agreed to version X, the system stores the exact text of the document they saw and agreed to. This provides a stronger audit trail; even if the consent document at version 1.0 is later revised or lost, the stored record contains precisely what the user agreed to. The tradeoff is that ConsentRecord rows can be

large. If the consent document is several kilobytes, and the platform has thousands of participants who re-sign across multiple versions, the table will grow accordingly.

The IP address handling is worth noting. The endpoint first checks the X-Forwarded-For header, taking the leftmost value (which represents the original client IP in a proxy chain), and falls back to request.client.host if that header is absent. The IP is then converted to packed binary format via ip_to_varbinary(), which uses Python's ipaddress module and handles both IPv4 (4 bytes) and IPv6 (16 bytes) correctly. Invalid IP strings are converted to None rather than raising an error. This binary representation is what is stored in ConsentRecord.IpAddress and is the same format used in the sessions table; the admin API decodes it back to dotted-decimal notation when displaying consent records.

The two database writes, the INSERT into ConsentRecord and the UPDATE to AccountData, are executed within a single transaction and committed together. This atomicity is important: it ensures there is never a state where a consent record exists but the account row hasn't been updated, or vice versa. If either write fails, both are rolled back and a 500 error is returned. After a successful commit, the response includes the new ConsentRecord ID, the version that was recorded, and accepted: true.

HCP API:

This API manages the relationship between Healthcare Providers (HCPs, RoleID = 3) and their patients (Participants, RoleID = 1). A link between the two is represented as a row in the HcpPatientLink table and can exist in one of four states: pending, active, rejected, or removed. All endpoints are accessible to RoleIDs 1, 3, and 4 unless otherwise noted, and all use effective_account_id and effective_role_id from the session, meaning they correctly respect the admin view-as system.

The Link Model and Status Lifecycle:

Before looking at the endpoints individually, it helps to understand the link lifecycle. A link begins as pending when either party initiates it. The other party then accepts it (moving it to active) or rejects it (moving it to rejected). Either party can remove an active link at any time, moving it to removed. A rejected or removed link is not deleted; the row is retained and the status updated in place. This means a link can be re-requested after rejection or removal: the request_link endpoint detects an existing row in a terminal state and updates it back to pending rather than inserting a new row.

This soft-delete approach preserves history and prevents duplicate rows in the table, but it also means the HcpPatientLink table can only ever have one row per HCP-patient pair. If a relationship is rejected, removed, and re-requested multiple times, there is no record of the earlier cycles; only the current state. Whether this is sufficient for the platform's audit requirements is worth considering.

Requesting a Link:

Either party can initiate a link request. The caller's role determines which side of the link they occupy: participants (RoleID = 1) become the patient, HCPs (RoleID = 3) become the HCP. Admins (RoleID = 4) are treated as participants for role-check purposes; the code explicitly remaps their role to 1 at the top of the handler before any validation runs. This means an admin using view-as as an HCP would still be treated as a participant when requesting a link, which may or may not be the intended behaviour. It is worth verifying how this interacts with the view-as system in practice.

The endpoint validates that the target account exists, is active, and has the complementary role: participants must target an HCP, and HCPs must target a participant. Attempting to link two participants or two HCPs returns a 400 error with a descriptive message. The requested_by field is set to "patient" or "hcp" based on who initiated, and this field is used by the respond endpoint to enforce that only the non-initiating party can accept or reject.

If an existing link is found in pending or active state, the request is rejected with a 409 Conflict. If the existing link is in rejected or removed state, it is re-activated to pending with an UPDATE rather than a new INSERT, preserving the original LinkID.

Listing Links:

This endpoint returns all links associated with the current user, with HCPs seeing rows where they are the HcpID and participants (and admins) seeing rows where they are the PatientID. An optional status query parameter filters results to a specific state. Results are ordered newest-first by RequestedAt.

The query joins AccountData twice, once for the HCP and once for the patient, to assemble display names using CONCAT(COALESCE(FirstName,''), ' ', COALESCE(LastName,'')). The COALESCE guards against null name fields, though the concatenation will still produce a leading or trailing space if only one name component is null. This is a cosmetic issue rather than a functional one, but worth noting if name display is important to the UI.

This is one of the few places in the codebase where a column name (where_col) is interpolated directly into the query string rather than passed as a parameter. This is safe because where_col is set to one of two hardcoded string literals ("l.HcpID" or "l.PatientID") determined entirely by the caller's role; it is never derived from user input. The status filter value, by contrast, is correctly passed as a parameterized argument.

Responding to a Link:

Only the party who did not initiate the link request can respond to it. The endpoint enforces this by checking the stored RequestedBy field against the caller's identity. If the HCP requested the link, only the patient can respond, and vice versa. Admins are handled as a special case: if an admin's account ID matches either the HcpID or PatientID on the link, they are treated as that party for the purpose of response eligibility.

The action field must be exactly "accept" or "reject"; any other value is rejected immediately at the top of the handler before any database access. Accepting sets the status to "active", rejecting sets it to "rejected".

When a link is accepted, the endpoint automatically creates a direct messaging conversation between the HCP and patient if one does not already exist. It checks for an existing direct-type conversation shared by both accounts using a self-join on ConversationParticipants, and only creates the new Conversations row and two ConversationParticipants rows if none is found. This is a side effect worth documenting: accepting a link request has consequences beyond just the HcpPatientLink table. If the conversation creation fails for any reason, the entire transaction is rolled back, so a link cannot be accepted without a corresponding conversation being created or confirmed to exist.

Consent Revocation and Restoration:

These two endpoints, POST /{link_id}/revoke-consent and POST /{link_id}/restore-consent,control the ConsentRevoked boolean on a link row. Only the patient or a system administrator can revoke or restore consent; HCPs have no access to these endpoints. Consent revocation does not change the link's Status field at all; the link remains active. The expectation is that other parts of the system check ConsentRevoked before granting the HCP access to patient data, though that enforcement logic is not present in this file.

Revocation and restoration are both reversible and can be toggled freely. There is no audit trail for individual revocation events; the UpdatedAt timestamp is updated each time, but only the current state is visible, not the history of changes. If the platform needs to track when consent was revoked and subsequently restored, an event log would need to be added.

The separation of revocation from link removal is meaningful: a patient who revokes consent is saying "I want this HCP to stop seeing my data" without necessarily ending the clinical relationship. A patient who removes the link entirely is ending both the data access and the relationship record. Both options being available gives patients more granular control.

Removing a Link:

Despite the HTTP method, this is not a hard delete; the row is retained and the Status is updated to "removed". Either party can remove a link, as can an admin. The endpoint verifies that the caller is one of the two parties on the link (or an admin) before allowing the update; attempting to remove a link you have no part in returns a 403. There is no guard against removing a link that is already in removed, pending, or rejected state, the UPDATE will succeed silently on any status, which is largely harmless but means a double-remove does not produce an error. The endpoint returns HTTP 204 (No Content) on success.

HCP Patients API:

This API is the read side of the HCP-patient relationship. Where hcp_links.py manages the existence and status of links between healthcare providers and their patients, this file handles what HCPs can actually see through those links: patient lists, completed surveys, and individual survey responses. It is restricted to RoleIDs 3 (HCP) and 4 (Admin), with participants having no access to any of these endpoints.

Access Control Architecture:

The central security mechanism in this file is the _verify_hcp_access() helper, which is called at the top of every data-access endpoint before any patient data is queried. It takes an HCP ID and a patient ID and checks the HcpPatientLink table for a row where both are present, Status = 'active', and ConsentRevoked = 0. If no such row exists, it raises an HTTP 403 immediately. This is a tight, single-purpose check that captures both the relationship requirement and the consent requirement in one query.

Admins (RoleID = 4) bypass this check entirely in all three endpoints; the call to _verify_hcp_access() is wrapped in a if caller_role != 4: guard each time. This means admins can view any patient's data regardless of whether a link or consent exists. This is consistent with the admin's system-wide access level, but it does mean admin access to patient health data is ungated by the consent model that governs HCP access. Whether this is appropriate depends on the platform's data governance policies and is worth documenting clearly.

It is also worth noting that _verify_hcp_access() takes the cursor as an argument rather than opening its own connection. This is good practice, it means the access check runs within the same database transaction as the data query that follows, using the same connection and cursor that the calling endpoint already has open, avoiding an unnecessary second connection.

Listing Patients:

For an HCP caller, this endpoint returns all patients linked to them where the link is active and consent has not been revoked. The result set includes the link ID, the patient's account ID, their display name (assembled from FirstName and LastName with COALESCE guards), and the date the link was established (RequestedAt, aliased as linked_since).

For admin callers, the behaviour branches based on whether the optional hcp_id query parameter is supplied. If it is, the same query runs but scoped to the specified HCP rather than the caller. If no hcp_id is given, the admin receives all active, consented links across every HCP on the platform, effectively a global view of all current HCP-patient relationships. This global query has no pagination, so on a large platform with many active links it could return a very large result set. If the platform scales significantly, adding limit and offset parameters to this endpoint would be worthwhile.

The linked_since field is sourced from RequestedAt rather than the timestamp at which the link was accepted. This means it reflects when the link was first requested, not when it became active. For a link that was pending for several days before acceptance, the displayed date may be earlier than when the relationship actually began. Whether this is the intended meaning of "linked since" is a product decision, but it is worth documenting.

Listing Patients Surveys:

After verifying access via _verify_hcp_access(), this endpoint returns a list of surveys that the patient has completed. It queries SurveyAssignment filtered to Status = 'completed' for the specified patient ID, joining Survey to get the title. Results are ordered by CompletedAt descending, so the most recently completed survey appears first.

Only completed surveys are visible, pending or expired assignments do not appear. This is appropriate for a clinical context, since an HCP is interested in actual results rather than surveys that are in progress or overdue. However, it also means an

HCP cannot see whether a patient has been assigned surveys they haven't yet completed, which might be relevant clinical information. If that visibility is needed in future, a status filter parameter could be added.

The response does not include the actual responses, it is an index of completed surveys that the HCP can then drill into using the responses endpoint below. The assignment_id and survey_id are both returned, giving the caller what they need to make the follow-up request.

Fetching Patient Responses:

This is the most data-sensitive endpoint in the file. After verifying the HCP's access, it retrieves every response the patient submitted for the specified survey, joining QuestionBank to include the question text and response type alongside the raw answer value. Responses are ordered by QuestionID, which corresponds to the order questions appear in the survey.

A 404 is returned if no responses are found for the given patient and survey combination. This covers two cases: the patient genuinely has no responses for that survey (perhaps it was assigned but never completed), and the survey ID itself doesn't exist. Both collapse to the same 404 response, which is the appropriate behaviour here since distinguishing them would reveal information about the survey's existence to an HCP who may not otherwise have access to it.

One thing to be aware of is that the access check does not verify that this particular survey was assigned to the patient through the HCP or that it is relevant to the clinical relationship. Any HCP with an active consented link to a patient can view that patient's responses to any survey on the platform, including surveys assigned by other researchers with no clinical connection to the HCP. Whether this breadth of access is intended is worth reviewing from a data minimisation perspective, a stricter implementation might limit HCP access to surveys explicitly related to their clinical context.

Health Tracking API:

File: backend/app/api/v1/health_tracking.py | Router prefix: /api/v1/health-tracking

The Health Tracking API enables participants to log structured health metrics over time, researchers to view k-anonymised population aggregates, and administrators to configure the metric catalogue dynamically. Nothing is hardcoded — every category and metric is stored in the database and managed via admin endpoints.

1.1 Enumerations

Two enumerations control how metric values are stored and how often participants are expected to log them.

health_tracking.py — Enumerations

```
# MetricType — governs input widget and value validation
class MetricType(str, Enum):
    number        = "number"       # free numeric (float)
    scale         = "scale"        # integer within [ScaleMin, ScaleMax]
    yesno         = "yesno"        # yes/no/true/false/1/0
    single_choice = "single_choice" # one value from ChoiceOptions list
    text          = "text"         # free-form string


# Frequency — shown to participant, not enforced server-side
class Frequency(str, Enum):
    daily    = "daily"
    weekly   = "weekly"
```

```
    monthly  = "monthly"

    any_time = "any"
```

1.2 Pydantic Response Models

MetricResponse and CategoryResponse are the primary read models returned to participants, HCPs, and researchers. MetricResponse is nested inside CategoryResponse so a single GET /metrics call returns the full catalogue grouped by category.

health_tracking.py — Read models

```
class MetricResponse(BaseModel):

    metric_id:      int

    category_id:    int

    metric_key:     str           # e.g. 'sleep_hours'

    display_name:   str

    description:    Optional[str]

    metric_type:    str           # MetricType enum value

    unit:           Optional[str]  # e.g. 'hours', 'glasses'

    scale_min:      Optional[int]

    scale_max:      Optional[int]

    choice_options: Optional[List[str]]  # JSON list for single_choice

    frequency:      str           # Frequency enum value

    display_order:  int
```

```
    is_active:      bool
    is_baseline:    bool
    is_deleted:     bool = False


class CategoryResponse(BaseModel):
    category_id:   int
    category_key:  str            # e.g. 'physical_health'
    display_name:  str
    description:   Optional[str]
    icon:          Optional[str]
    display_order: int
    is_active:     bool
    is_deleted:    bool = False
    metrics:       List[MetricResponse]  # nested
```

1.3 Entry Submission Models

Participants submit entries in batches. Each individual entry in the batch is validated by EntrySubmit. The outer BatchEntrySubmit carries the is_baseline flag so that initial onboarding submissions can be marked as baseline snapshots for before-vs-after housing analysis.

health_tracking.py — Entry submission models

```
class EntrySubmit(BaseModel):
```

```
    metric_id:  int
    value:      str        # sanitized_string() applied via field_validator
    notes:      Optional[str] = None
    entry_date: date = None # defaults to today() server-side


class BatchEntrySubmit(BaseModel):
    entries:     List[EntrySubmit]
    is_baseline: bool = False
```

1.4 Participant Endpoints

Role 1 (participant) and Role 4 (admin) may call these endpoints. All participant data is scoped to the authenticated user's effective_account_id so participants cannot read or write each other's entries.

Method	Path	Auth (RoleID)	Description
GET	/api/v1/health-tracking/metrics	1, 4	Active categories + nested metrics. Pass ?lang=fr for French translations.
POST	/api/v1/health-tracking/entries	1, 4	Batch upsert entries for one or more metrics. UNIQUE(ParticipantID,MetricID,Entr

			yDate) — submitting the same date overwrites.
GET	/api/v1/health-tracking/entries	1, 4	Own entries. Filter: start_date, end_date, metric_id, category_key.
GET	/api/v1/health-tracking/history/{metric_id}	1, 4	Full time-series for one metric (for charting).
GET	/api/v1/health-tracking/baseline	1, 4	All entries with IsBaseline=1 for this participant.

Upsert Pattern — POST /entries

Each entry is independently validated then written with MySQL's ON DUPLICATE KEY UPDATE to guarantee idempotency. Resending the same (participant, metric, date) overwrites Value and Notes without creating a duplicate row.

health_tracking.py — Upsert logic (submit_entries)

```
# Per-entry validation
error = _validate_entry_value(
    entry.value, metric['MetricType'],
    metric['ScaleMin'], metric['ScaleMax'], choice_options
```

```
)
if error:
    raise HTTPException(422, detail=error)


# Upsert — UNIQUE(ParticipantID, MetricID, EntryDate)
cur.execute(
    """
    INSERT INTO TrackingEntry
        (ParticipantID, MetricID, Value, Notes, EntryDate, IsBaseline)
    VALUES (%s, %s, %s, %s, %s, %s)
    ON DUPLICATE KEY UPDATE
        Value = %s,
        Notes = %s
    """,
    (participant_id, entry.metric_id, entry.value, entry.notes,
     entry_date, int(body.is_baseline),
     entry.value, entry.notes),
)
```

Value Validation — _validate_entry_value()

Before each INSERT the server validates the submitted value matches the declared MetricType. Invalid values return HTTP 422 with an explanatory message before any database write is attempted.

health_tracking.py — _validate_entry_value()

```
def _validate_entry_value(value, metric_type, scale_min, scale_max,
choice_options):

    if metric_type == 'number':

        float(value)   # raises ValueError → HTTP 422


    elif metric_type == 'scale':

        n = int(value)

        assert scale_min <= n <= scale_max


    elif metric_type == 'yesno':

        assert value.lower() in {'yes','no','true','false','1','0'}


    elif metric_type == 'single_choice':

        assert value in (choice_options or [])


    # 'text' — any sanitized string is accepted

    return None  # no error
```

1.5 Admin Endpoints

Role 4 (admin) manages the metric catalogue. Categories and metrics support soft-delete (IsDeleted flag) so historical entries are never orphaned. Reorder endpoints accept a full ordered list of IDs and apply them atomically in a loop.

Method	Path	Auth (RoleID)	Description
GET	/api/v1/health-tracking/admin/categories	4	All categories (active + inactive) with metric counts.
POST	/api/v1/health-tracking/admin/categories	4	Create a category. category_key must be unique.
PUT	/api/v1/health-tracking/admin/categories/{id}	4	Update name, description, icon, display order, is_active.
GET	/api/v1/health-tracking/admin/metrics	4	All metrics with their parent category.
POST	/api/v1/health-tracking/admin/metrics	4	Create a metric under a category.
PUT	/api/v1/health-tracking/admin/metrics/{id}	4	Update metric fields.
PATCH	/api/v1/health-tracking/admin/metrics/{id}/toggle	4	Toggle IsActive for a single metric.

PUT	/api/v1/health-tracking/admin/metrics/reorder	4	Bulk-reorder: accepts ordered list of metric IDs.

All string fields (category_key, display_name, description, icon, unit, choice_options items) are passed through sanitized_string() via Pydantic field_validators before any database write.

1.6 Researcher Endpoints

Role 2 (researcher) and Role 4 (admin) access aggregate and export endpoints. K-anonymity is enforced: data points are suppressed when the number of distinct participants contributing to that point falls below the k_anonymity_threshold system setting (default 5). Participant IDs are SHA-256 hashed in CSV exports.

Method	Path	Auth (RoleID)	Description
GET	/api/v1/health-tracking/research/aggregate	2, 4	Daily avg/min/max per metric, k-anon filtered. Params: metric_id, start_date, end_date.
GET	/api/v1/health-tracking/research/export	2, 4	Streaming CSV. Params: metric_ids (comma-separated), start_date, end_date.

			Participant IDs are SHA-256 hashed.
GET	/api/v1/health-tracking/research/categories	2, 4	Per-category summary: entry count, distinct participants, date range. k-anon filtered.

K-Anonymity Enforcement

The k_anonymity_threshold is read from SystemSettings on every request via get_int_setting() which has a 30-second in-memory cache. The HAVING clause below prevents any date bucket from being returned when fewer than k distinct participants contributed entries.

health_tracking.py — K-anonymity pattern (research/aggregate)

```
k = get_int_setting('k_anonymity_threshold')  # from SystemSettings, cached
30 s


query = '''
    SELECT
        te.EntryDate,
        COUNT(DISTINCT te.ParticipantID) AS participant_count,
        AVG(CAST(te.Value AS DECIMAL(10,2))) AS avg_value,
```

```
        MIN(CAST(te.Value AS DECIMAL(10,2))) AS min_value,
        MAX(CAST(te.Value AS DECIMAL(10,2))) AS max_value
    FROM TrackingEntry te
    WHERE te.MetricID = %s
    GROUP BY te.EntryDate
    HAVING participant_count >= %s
    ORDER BY te.EntryDate ASC
"""
cur.execute(query, (metric_id, k))
```

CSV Export — Participant ID Hashing

The export endpoint streams a CSV directly to the HTTP response using FastAPI's StreamingResponse. Participant IDs are irreversibly replaced with the first 16 hex characters of their SHA-256 digest before any row reaches the response buffer.

health_tracking.py — CSV export with hashed IDs

```
import hashlib, csv, io
from fastapi.responses import StreamingResponse


def _hash_id(participant_id: int) -> str:
    return hashlib.sha256(str(participant_id).encode()).hexdigest()[:16]
```

```
# Inside export endpoint:

output = io.StringIO()

writer = csv.writer(output)

writer.writerow(['participant_hash','metric_key','entry_date','value','is_baseline'])

for row in rows:

    writer.writerow([

        _hash_id(row['ParticipantID']),

        row['MetricKey'],

        row['EntryDate'],

        row['Value'],

        row['IsBaseline'],

    ])

output.seek(0)

return StreamingResponse(

    iter([output.read()]),

    media_type='text/csv',

    headers={'Content-Disposition': 'attachment;
filename=health_tracking_export.csv'},

)
```

1.7 Database Schema

Three new tables were added in migration 009_add_health_tracking.sql, plus optional translation tables.

migration 009_add_health_tracking.sql — table definitions

```
-- Admin-configurable tracking categories
CREATE TABLE TrackingCategory (
    CategoryID   INT AUTO_INCREMENT PRIMARY KEY,
    CategoryKey  VARCHAR(64) UNIQUE NOT NULL,
    DisplayName  VARCHAR(128) NOT NULL,
    Description  TEXT,
    Icon         VARCHAR(64),
    DisplayOrder INT DEFAULT 0,
    IsActive     TINYINT(1) DEFAULT 1,
    IsDeleted    TINYINT(1) DEFAULT 0,
    CreatedAt    DATETIME DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt    DATETIME ON UPDATE CURRENT_TIMESTAMP
);

-- Metrics linked to categories
CREATE TABLE TrackingMetric (
    MetricID     INT AUTO_INCREMENT PRIMARY KEY,
```

```
    CategoryID   INT NOT NULL REFERENCES
TrackingCategory(CategoryID),

    MetricKey    VARCHAR(64) UNIQUE NOT NULL,

    DisplayName  VARCHAR(128) NOT NULL,

    MetricType   ENUM('number','scale','yesno','single_choice','text'),

    Unit         VARCHAR(32),

    ScaleMin     INT DEFAULT 1,

    ScaleMax     INT DEFAULT 10,

    ChoiceOptions JSON,

    Frequency    ENUM('daily','weekly','monthly','any') DEFAULT 'daily',

    DisplayOrder INT DEFAULT 0,

    IsActive     TINYINT(1) DEFAULT 1,

    IsBaseline   TINYINT(1) DEFAULT 0,

    IsDeleted    TINYINT(1) DEFAULT 0,

    CreatedAt    DATETIME DEFAULT CURRENT_TIMESTAMP

);


-- Participant entries — one row per participant per metric per date

CREATE TABLE TrackingEntry (

    EntryID       INT AUTO_INCREMENT PRIMARY KEY,

    ParticipantID INT NOT NULL REFERENCES AccountData(AccountID),
```

```
    MetricID      INT NOT NULL REFERENCES TrackingMetric(MetricID),
    Value         TEXT NOT NULL,
    Notes         TEXT,
    EntryDate     DATE NOT NULL,
    IsBaseline    TINYINT(1) DEFAULT 0,
    CreatedAt     DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_entry (ParticipantID, MetricID, EntryDate)
  );
```

Optional translation tables TrackingCategoryTranslation and TrackingMetricTranslation store French (fr) and Spanish (es) overrides for DisplayName and Description. The GET /metrics endpoint applies translations automatically when ?lang= is passed.

1.8 Localization

All participant-facing metric display names and category names are localizable. Translations are stored in the database (TrackingMetricTranslation, TrackingCategoryTranslation) rather than ARB files so they can be updated at runtime by admins without a deployment. ARB files carry UI chrome strings (tab labels, button labels, error messages) for the frontend.

Relevant ARB keys added: healthTrackingTitle, healthTrackingLogToday, healthTrackingHistory, healthTrackingBaseline, healthTrackingBaselinePrompt, healthTrackingSaved, trackingCategoryPhysicalHealth, trackingCategoryMentalHealth, (and equivalents for all 10 categories).

Messaging API:

This API implements the platform’s internal messaging system, covering direct conversations between users and the friend request mechanism that governs who participants can message. The system is permission-gated at two levels: who is allowed to start a conversation with whom (enforced by _can_message()), and who is allowed to read or write within a conversation (enforced per-endpoint by checking ConversationParticipants).

Permission Model:

The _can_message() helper is the central policy function for messaging authorization. It is worth understanding clearly because it encodes the social architecture of the entire platform.

Admins (RoleID = 4) can message anyone without restriction. HCPs (RoleID = 3) can only message participants with whom they have an active, consent-given link in HcpPatientLink, the same check as _verify_hcp_access() in hcp_patients.py, though implemented inline here rather than through that shared helper. Participants (RoleID = 1) can message either their linked HCP (again requiring an active, non-revoked link) or any participant they have an accepted friend request with. Researchers (RoleID = 2) can only message other active researchers, and no friend request or link is required, the entire researcher population is mutually messageable by default.

This function is only called during conversation creation. Once a conversation exists and both parties are in ConversationParticipants, subsequent messages are gated only by participation membership, not by the current link or friendship status. This means that if a patient revokes consent after a conversation has already been created, the HCP can still send and read messages in that conversation, the revocation does not retroactively block messaging. Whether this is intended behaviour is an important question for the platform's data governance policy.

Conversation Creation:

This endpoint starts a new direct conversation between the caller and a target account, or returns the existing one if it already exists. Before doing either, it verifies that the target account exists and is active, and that _can_message() returns true for the caller-target pair.

The actual conversation lookup and creation is delegated to _get_or_create_conversation(), which searches ConversationParticipants for a direct-type conversation shared by both accounts. This is the same logic used in hcp_links.py when a link is accepted, the two flows are parallel and will correctly detect conversations created by either path. If a conversation already exists, the endpoint returns the existing conv_id with "created": false. If a new one is created, three rows are inserted: one into Conversations and one each into ConversationParticipants for the caller and the target.

The _get_or_create_conversation() helper calls db.commit() internally when it creates a new conversation, which is a slight inconsistency with the rest of the codebase where commits are usually done in the calling handler. This means the commit happens inside the helper rather than being controlled by the endpoint. It is not harmful, but maintainers should be aware of it if the helper is ever refactored or reused.

Listing Conversations:

This endpoint, GET /messages/conversations, returns all conversations the caller participates in, along with a preview of the last message and the other participant's display name. The query is worth understanding because it is the most complex in the file.

The query joins ConversationParticipants twice, once to find conversations the caller belongs to, and again (with cp2.AccountID != caller_id) to find the other participant in each conversation. It then does a left join against a subquery that finds the most recent message per conversation by selecting the row with the maximum MessageID. Using MAX(MessageID) as a proxy for recency assumes that message IDs are

assigned in insertion order, which is true for auto-increment primary keys but is worth noting as an implicit assumption. Results are ordered by the last message timestamp, falling back to the conversation creation time if no messages have been sent yet, so empty conversations still appear in the list rather than being lost.

The query is designed for two-party direct conversations and would produce unexpected results for group conversations, if those were ever added. The current schema supports them (the ConvType field exists), but the query logic assumes exactly one other participant per conversation.

Reading and Sending Messages:

Both endpoints, GET and POST /messages/conversations/{conv_id}/messages, share the same first step: verifying that the caller has a row in ConversationParticipants for the given conv_id. If they do not, a 403 is returned. This is the post-creation access check, and unlike _can_message(), it makes no reference to links or friendships, it only checks participation membership. This means access to a conversation is stable once granted, regardless of subsequent changes to the underlying relationship.

Reading returns all messages in the conversation ordered by SentAt ascending (oldest first), which is the natural chronological order for a chat display. Each message includes the sender's display name joined from AccountData.

Sending inserts a new row into Messages and then immediately re-fetches the SentAt timestamp that the database assigned, returning it to the caller. This round-trip is necessary because SentAt is set by MySQL's DEFAULT CURRENT_TIMESTAMP rather than being supplied by the application, the application doesn't know what value was used until it queries back. The fallback to datetime.utcnow() if the re-fetch fails is defensive but should never be reached in practice. Message bodies are sanitized by the SendMessageRequest validator before reaching the handler.

Friend Requests:

The friend request system applies only to participants (RoleID = 1) and admins. Researchers, HCPs, and the messaging relationships between them are handled through other mechanisms (mutual researcher visibility and HCP-patient links respectively).

Sending a request; POST /messages/friend-request accepts a target email address and always returns the same 202 response regardless of whether the email belongs to an active participant account. This is a deliberate privacy measure, a caller cannot use this endpoint to discover whether a given email address is registered on the platform. The lookup internally restricts to RoleID = 1 and IsActive = 1, so requests to HCP, researcher, or admin emails silently do nothing. Duplicate requests are handled with INSERT IGNORE, so re-sending a request to someone you've already requested also silently succeeds. The email field is sanitized before use.

Listing incoming requests; GET /messages/friend-requests/incoming returns pending requests where the caller is the target. HCPs and researchers are short-circuited at the top of the handler and always receive an empty list, since those roles don't use the friend system.

Responding to a request; PUT /messages/friend-requests/{request_id}/respond accepts "accept" or "reject" as the action. Only the TargetAccountID on the request may respond, the requester cannot accept their own request. Admins can respond on behalf of any target, which is consistent with their system-wide access but means an admin could accept a friend connection between two participants without either consenting. The endpoint guards against responding to already-processed requests by checking that the current status is "pending" before updating. Accepting a friend request does not automatically create a conversation, the caller must separately call the create conversation endpoint if they want to start messaging. This is a different behaviour from the HCP link acceptance flow, which creates a conversation automatically.

Listing friends; GET /messages/friends returns the caller's messaging contacts, but the implementation differs meaningfully by role. Researchers receive every other

active researcher on the platform in a single query, no friend request history is involved. Participants and admins receive only accounts they have an accepted FriendRequests row with. HCPs receive an empty list unconditionally, since their contacts are managed through HcpPatientLink rather than this system. The friend list query for participants uses a CASE expression in both the SELECT and the JOIN ON clause to resolve which side of the friend request the other party is on, handling both the case where the caller was the requester and where they were the target.

Researcher search; GET /messages/researchers/search allows researchers and admins to search for other researchers by first or last name. The query parameter q must be at least two characters, and results are limited to 20. The search uses LIKE %q% on both FirstName and LastName independently, which means a two-word query like "John Smith" would not match anyone, only single-name fragments are supported. The response returns only account_id and display_name, deliberately omitting email addresses to limit information exposure within the researcher population.

Participant API:

This API handles everything a participant interacts with on the survey side: browsing their assigned surveys, loading questions, saving drafts, submitting answers, and viewing their responses compared against aggregate data. All endpoints are restricted to RoleIDs 1 (Participant) and 4 (Admin), with admins again respecting the view-as system via effective_account_id.

Draft Save and Load:

The draft system allows participants to save in-progress answers before submitting. Drafts are stored as a JSON blob in the DraftData column of SurveyAssignment rather than in the Responses table. This is a deliberate separation; draft data is transient and submission-scoped, while Responses rows are the permanent record of completed answers.

The save endpoint serializes the submitted question_responses list into a dictionary keyed by question_id (as a string, since JSON keys are always strings) and writes the resulting JSON to DraftData. It first checks that the assignment exists and then silently returns without saving if the assignment is already completed, saving a draft over a completed survey is treated as a no-op rather than an error, which is a reasonable UX choice since the submission has already been recorded. The endpoint returns HTTP 204 (No Content) on success.

The load endpoint reads DraftData back and parses it from JSON, returning it under a "draft" key. If no draft has been saved, it returns {"draft": {}} rather than 404 or null, making it safe for the frontend to call unconditionally when loading a survey form. Draft data is cleared when the survey is submitted successfully, so a participant resuming a completed survey will see an empty draft object.

Survey Submission:

This endpoint runs a sequence of validation checks before writing any data, and each check is worth understanding individually.

First, it confirms the survey exists and has PublicationStatus = 'published'. A survey in draft or closed state cannot receive submissions. Second, it confirms the participant has an assignment for this survey; unassigned participants receive a 403 rather than a 404, which avoids revealing whether the survey exists at all. Third, it checks the assignment's current status: if it is already completed, a 400 is returned; if it is expired, another 400 is returned. Fourth, if a DueDate exists on the assignment, the current time is fetched from the database using SELECT UTC_TIMESTAMP() rather than from Python's clock, this is the correct approach since it avoids timezone mismatches between the application server and the database. If the due date has passed, the assignment is updated to expired and committed before the 400 error is raised, ensuring the status is persisted even though the submission is rejected. Fifth, the endpoint checks whether any Responses rows already exist for this participant and survey, returning 400 if they do. This is a belt-and-suspenders guard against double submission, since the assignment status check above should catch most cases.

Response value normalisation is the most involved part of the submission logic. For single_choice questions, the frontend sends an OptionID as a string (e.g. "63"). The endpoint validates that this ID belongs to the correct question and then replaces it with the option's OptionText before storing. This means the Responses table stores human-readable text, not raw IDs, which makes the data more portable and meaningful outside the application. For multi_choice questions, the frontend sends either a JSON array of option IDs (e.g. ["32","33"]) or a legacy comma-separated string (e.g. "31,34,36"). The endpoint tries JSON parsing first and falls back to the comma-separated format, then validates each ID against QuestionOptions, and stores the result as a comma-separated string of option text values in the same DisplayOrder sequence. If any option ID is invalid, the entire submission fails with a 400 error. For all other response types (text, numeric, etc.) the value is stored as-is.

The ON DUPLICATE KEY UPDATE clause on the Responses insert is worth noting. This means if somehow a response row already exists for a given participant-survey-question combination, the insert will update it rather than failing.

Combined with the pre-check for existing responses, this should rarely fire, but it prevents a hard failure if the pre-check ever races with a concurrent submission.

After all responses are inserted, the assignment is updated to status = 'completed', CompletedAt = UTC_TIMESTAMP(), and DraftData = NULL in a single statement. The draft is cleared atomically with the completion, so there is no window where a completed assignment still has stale draft data attached.

The error handling here is slightly different from the rest of the codebase. HTTPException is caught and re-raised with a rollback, and a bare except Exception block catches everything else and wraps it in a 500 error. This broad catch is a deviation from the pattern in other files, which generally only catch mysql.connector.Error. It is less precise but more robust against unexpected errors mid-submission.

Listing Surveys:

This endpoint returns a list of all surveys assigned to the current participant, with optional filtering via include_completed and include_expired query parameters (both default to true). The result always includes pending assignments, with completed and expired ones added to the IN clause based on the filter parameters. The DraftData column is not returned directly; instead it is coerced to a boolean has_draft field, so the frontend knows a draft exists without receiving the potentially large JSON blob.

The ORDER BY clause is multi-level and deliberate. Pending surveys appear first (the (sa.Status = 'pending') DESC expression evaluates to 1 for pending and 0 for others, sorting pending to the top). Within pending surveys, those with a due date sort before those without (sa.DueDate IS NULL evaluates to 1 for null, pushing null dates to the end), and then by due date ascending so the most urgent appears first. Completed and expired surveys then appear ordered by assignment date descending. This ordering is designed to surface the most actionable items at the top of the list.

Responded Surveys with Answers:

This endpoint is marked in a comment as "potentially temporary" and returns a detailed view of all surveys the participant has actually submitted responses to, including the question list and the participant's answers for each. It is more data-intensive than the list endpoint and uses a three-query pattern to avoid N+1 problems: one query to find all relevant surveys, one to fetch all questions for those surveys, and one to fetch all responses. The results are then joined in Python using a (survey_id, question_id) dictionary as a lookup map.

Questions with no corresponding response entry will have response_value = None in the output, which can happen if a question was added to the survey after the participant submitted. This is a legitimate edge case to be aware of if the survey schema ever changes post-submission.

The endpoint uses a LEFT JOIN to SurveyAssignment rather than an inner join, meaning it will return surveys the participant has responses for even if their assignment row no longer exists. This could occur if an assignment was deleted after completion. Whether those orphaned responses should still be visible is a product decision.

Response Comparison:

This endpoint places the participant's individual answers alongside aggregate statistics for the same survey, intended for showing participants how their responses compare to the group. Before returning any data, it verifies that the participant has an assignment for the survey; this prevents a participant from probing aggregate data for surveys they were never assigned to.

The aggregate data is generated by AggregationService.get_survey_aggregates(), which is defined in a separate service module and handles k-anonymity suppression. When the number of respondents for a given question falls below the suppression threshold (K = 5, as noted in the chart-data endpoint below), the aggregate is withheld to prevent re-identification of individual

responses. The endpoint returns the participant's own answers regardless of suppression; the suppression only applies to the aggregate side.

There is a commented-out block that would return a 400 error if the participant has not yet submitted any responses. This check is currently disabled, meaning the endpoint can be called before submission and will return an empty participant_answers list alongside the aggregates. Whether this is intentional or an oversight is worth confirming; allowing pre-submission access to aggregate data might be considered a design issue if participants are supposed to answer independently before seeing group results.

Chart Data:

This is similar to the compare endpoint but structured differently for frontend charting. Rather than separating participant answers and aggregates into two top-level fields, it interleaves them per question; each ChartQuestionData item contains both the participant's response_value and the aggregate statistics for that question. Optional category and response_type query parameters allow the frontend to request data for a subset of questions, which is useful for rendering category-specific charts without fetching the full survey.

The k-anonymity suppression is handled here explicitly at the response-building stage. For each question, if the aggregate data carries suppressed: true, the aggregate field in the response is set to None rather than the suppressed data, and the suppressed boolean is passed through so the frontend can display an appropriate message rather than a missing chart. This makes suppression visible and actionable for the user.

The question filter clauses (category and response_type) are the one place in this file where column names from filter parameters are interpolated into the query string via f-string. However, these values are only inserted into the WHERE clause as SQL condition fragments; the actual filter values themselves are still passed as parameterized arguments (%s). The column names being compared against

(qb.Category and qb.ResponseType) are hardcoded strings in the application code, not derived from user input, so there is no injection risk here.

Question Bank API:

This file provides full CRUD management for the question bank; the central library of reusable questions that researchers draw from when building surveys and templates. All endpoints are restricted to RoleIDs 2 (Researcher) and 4 (Admin) via a router-level dependency. The question bank is shared across the platform, so any researcher can see and potentially edit any question, which is worth bearing in mind if question ownership ever becomes a requirement.

Categories:

This endpoint, GET /questions/categories, returns the full list of question categories from the QuestionCategories table, ordered by DisplayOrder. Categories are used to group related questions and can be filtered on in the list endpoint. The response includes a category_key rather than a display name, implying that human-readable labels are handled by the frontend (likely via translation). This endpoint has no authentication guard beyond the router-level role requirement, so any researcher or admin can retrieve the category list.

Creating a Question:

This uses the POST /questions/ endpoint. When a new question is created, the handler first inserts the core QuestionBank row with all fields including the scale bounds, then handles options and translation as separate steps. Options are only created if both question.options is non-null and the response type is single_choice or multi_choice; supplying options for a number or yesno question is silently ignored rather than rejected, which may be worth tightening if the API is used by external clients who might inadvertently send options for the wrong question type.

After the question and its options are written, translate_question() and translate_options() are called automatically. These functions, defined in a separate translation service module, generate translations into all supported languages and store them in a translations table. This means every question created through this API is immediately available in all supported languages without any additional manual step.

The translations are committed in the same connection as the question data. If translation fails, for example, if an external translation service is unavailable, the commit will not include the translations, but the question itself will already have been committed in the previous step, leaving it without translations until the next update triggers a re-translation.

The response is built by re-fetching the newly created row rather than constructing it from the input data, which ensures the response accurately reflects what was stored, including any database-side defaults or timestamps.

Listing Questions:

This utilizes the GET /questions/ endpoint. The list endpoint supports optional filtering by category and response_type as query parameters, with pagination via limit (1–1000, default 500) and offset. Both filter values are passed as parameterized arguments. For each question in the result set that has a single_choice or multi_choice response type, a second query is issued to fetch the associated options via get_question_options(). This means the endpoint issues one query per choice question in the result set, which is an N+1 pattern. At a default limit of 500 questions, this could be a significant number of round trips on a large question bank. If performance becomes a concern, this could be replaced with a single joined query that fetches all options for all returned questions in one go and groups them in Python.

Fetching and Updating Individual Questions:

This uses the GET /questions/{question_id} and PUT /questions/{question_id}. The single-question fetch at GET /questions/{question_id} follows the same pattern as the list endpoint: fetch the base row, then conditionally fetch options if the type requires them. It additionally returns CreatedAt and UpdatedAt timestamps, which the list endpoint omits.

The update endpoint at PUT /questions/{question_id} builds its UPDATE statement dynamically from whichever fields are present in the request body, using the same pattern seen in the assignment update endpoint; an updates list and a params list are built up conditionally, and the query is only executed if at least one field was supplied. If no fields are provided, the database is not touched and the current record is re-fetched and returned unchanged. This is a valid no-op path but, as noted in earlier documentation, a 422 might be more informative here.

Option updates during a PUT are handled with a full replace strategy: if question.options is present in the request body (even as an empty list), all existing options are deleted via delete_question_options() and the new set is inserted. Passing an empty list therefore clears all options, which is the correct behaviour for changing a question from multi_choice to a type that has no options, but could be surprising if a caller sends a partial option list expecting a merge. There is no patch-style option update; it is always a full replacement.

Re-translation is triggered if either title or question_content changed, and option translations are updated if the options list was replaced. The translation calls happen after the updated data has been committed, so if translation fails, the question content update is preserved but the translation table may be stale until the next edit.

Deleting a Question:

The delete endpoint at DELETE /questions/{question_id} takes care to handle the question's relationships before removing the row itself. It explicitly unlinks the question from any surveys via QuestionList and from any templates via TemplateQuestions, ensuring those survey and template definitions are not left with dangling references. Existing Responses rows that reference this question are not deleted; instead, QuestionID is set to NULL, preserving the response data (the participant's submitted value) while severing the link to the question definition that no longer exists. This mirrors the approach taken in the deletion request approval flow and reflects a general platform philosophy of retaining submitted data rather than cascading deletes through response history.

The QuestionOptions rows are removed via database-level CASCADE when the QuestionBank row is deleted, rather than through an explicit DELETE statement in the handler. This is noted in the docstring. Translation rows, if any exist in a translations table, are not explicitly deleted here; whether those are also handled by a cascade or left as orphans depends on the foreign key configuration of the translations table, which is defined in the schema rather than visible in this file and should be verified.

Research API:

This file provides the researcher-facing analytics layer of the platform. Its endpoints expose survey response data in two forms: per-survey views for drilling into a single survey's results, and cross-survey views for querying across the entire data bank at once. All endpoints require RoleID 2 (Researcher) or 4 (Admin). Critically, this file contains almost no database logic of its own, the heavy lifting is delegated entirely to AggregationService, a shared service class instantiated once at module level as _aggregation. The endpoints here are essentially thin wrappers that call into that service, validate the result, and shape it into a Pydantic response model.

Privacy Architecture:

Two privacy mechanisms run across every data endpoint in this file and are worth understanding before looking at individual endpoints.

The first is participant anonymisation. Individual response rows never expose account IDs or any directly identifying information. Instead, each participant is represented by an anonymous_id, which is a SHA-256 hash of their participant ID. This is a one-way transformation that allows a researcher to track that the same participant answered multiple questions within a dataset (because the same ID produces the same hash) without being able to reverse it back to an account. The hashing is performed inside AggregationService rather than in this file.

The second is k-anonymity suppression. Whenever fewer than five distinct participants have responded to a question or survey, the aggregate data for that question is withheld and a suppressed: true flag is returned in its place, along with an optional reason string. This threshold prevents a researcher from inferring an individual's response from group statistics when the group is too small. The suppression check is also handled inside AggregationService, and the response models in this file include suppressed and reason fields precisely to carry that signal through to the caller.

Per-Survey Endpoints:

The survey list is available at GET /surveys, which returns every survey on the platform alongside its publication status, the count of distinct respondents, and the number of questions. Respondent count is computed with COUNT(DISTINCT r.ParticipantID) in a LEFT JOIN against Responses, so surveys with no responses appear in the list with a count of zero rather than being excluded. Question count is computed via a correlated subquery against QuestionList. This is the one endpoint in the file that queries the database directly rather than going through AggregationService.

High-level statistics for a single survey are available at GET /surveys/{survey_id}/overview, which returns respondent count, completion rate, question count, and suppression status via _aggregation.get_survey_overview(). If the service returns None, the endpoint raises a 404, this is the standard null-check pattern used throughout the file for all service calls.

Individual anonymised response rows are available at GET /surveys/{survey_id}/responses. This endpoint accepts optional category and response_type query parameters to scope the response columns, which is useful when a researcher only wants to examine a subset of questions. The response includes a questions list describing the columns and a rows list of anonymous_id / responses pairs, where responses is a dictionary keyed by question ID. If the survey has fewer than five respondents, the rows list will be empty and suppressed will be true, researchers cannot retrieve individual rows from small-sample surveys even in anonymised form.

Aggregate statistics per question are available at GET /surveys/{survey_id}/aggregates, returning a list of QuestionAggregateResponse objects, each containing the computed statistics in a data field whose shape varies by response type (a histogram for number questions, choice distribution percentages for single_choice and multi_choice, etc.). An individual question's aggregate can be fetched in isolation at GET /surveys/{survey_id}/aggregates/{question_id}, which is useful when

the frontend only needs to render a single chart without fetching the full survey's worth of aggregates.

CSV export for a single survey is available at GET /surveys/{survey_id}/export/csv. The endpoint calls _aggregation.get_survey_overview() first to retrieve the survey title, which is then used to construct a sanitised filename, spaces and forward slashes are replaced with underscores, producing something like Blood_Pressure_Study_responses.csv. The CSV content itself is generated by _aggregation.get_individual_csv_export() and returned as a StreamingResponse with Content-Disposition: attachment, prompting the browser to download the file rather than display it. The same category and response_type filters available on the responses endpoint are also accepted here.

Cross-Survey Endpoints:

The cross-survey endpoints operate on the full data bank rather than a single survey. All of them accept an optional survey_ids list parameter; if omitted, the query spans all surveys on the platform. This is a powerful but potentially expensive operation on a large dataset; there is no pagination on any of these endpoints, which means a query across all surveys and all questions could return a very large response. If the platform grows significantly, scoping by survey_ids or adding pagination would be worth considering.

An overview of the data bank is available at GET /cross-survey/overview, returning per-survey respondent counts, total unique respondent count across all selected surveys, total question count, and an average completion rate. The suppression flag here reflects whether the overall dataset meets the k-anonymity threshold.

The field picker for building a custom cross-survey query is served by GET /cross-survey/questions, which returns a flat list of all questions available across the selected surveys. Each question includes its survey_id and survey_title alongside its content and metadata, so the frontend can group or label questions by their source

survey. Optional category and response_type filters allow the list to be scoped before the researcher selects specific fields.

Individual anonymised rows spanning multiple surveys are available at GET /cross-survey/responses. Beyond the survey_ids filter, this endpoint also accepts a question_ids list to scope results to specific fields, and date_from / date_to parameters to filter by when responses were submitted. The response includes a suppressed_surveys list identifying any individual surveys within the selection whose data was withheld due to insufficient respondents, allowing the frontend to communicate partial suppression clearly. Aggregates across the data bank, following the same structure as the per-survey aggregate endpoint, are available at GET /cross-survey/aggregates with the same set of filters.

Cross-survey CSV export is available at GET /cross-survey/export/csv and accepts the same filtering parameters as the responses endpoint. Unlike the per-survey CSV endpoint, the filename is hardcoded to data_bank_export.csv rather than being derived from the selection; since the export may span many surveys, there is no single title to use. If the service returns None (indicating no matching surveys were found), the endpoint raises a 404.

Responses API:

This is a small but security-sensitive file containing a single endpoint, POST /, that handles participant survey submission. It is restricted exclusively to RoleID 1 (Participant), unlike most other endpoints in the codebase, admins are not included in the role check here, meaning there is no way for an admin to submit responses on behalf of a participant even through the view-as system. Whether this is intentional or an oversight is worth confirming, since it creates an asymmetry with other participant-facing endpoints.

It is also worth noting that this file represents an older or parallel submission path alongside the equivalent endpoint in participant_surveys.py. The two overlap in purpose but differ in behaviour in several important ways, which are discussed below.

The Submission Endpoint:

The file defines two standalone utility functions and a validation helper that together form the submission's type-checking layer, separate from Pydantic.

_validate_response_value() checks that a submitted string value is appropriate for its question's response type. For number questions it attempts a float() conversion, returning an error message if it fails. For scale questions it does the same but also enforces that the result falls between 1 and 10; notably this uses a hardcoded range rather than reading ScaleMin and ScaleMax from QuestionBank, so if a researcher creates a scale question with a non-default range, this validator will incorrectly reject values outside 1–10 even if they are within the question's intended range. This is a bug worth addressing if non-default scale ranges are in use. For yesno questions, it checks the lowercased value against the set {"yes", "no", "1", "0", "true", "false"}, which is a permissive set that accepts several representations. For single_choice and multi_choice questions, it validates the submitted value against the list of OptionText values fetched from QuestionOptions. For openended questions, any non-empty string is accepted.

_parse_multi_choice() handles the two formats in which a multi-choice response may arrive: a JSON array string (e.g. '["Option A", "Option B"]') or a legacy

comma-separated string (e.g. "Option A,Option B"). It tries JSON parsing first and falls back to the comma-separated split if that fails, which matches the same dual-format support in participant_surveys.py. This parsing function is shared between validation and normalisation.

_normalize_response_value() converts any multi_choice response to a canonical JSON array before storage, regardless of which format it arrived in. This means the Responses table always stores multi-choice answers as JSON arrays for this submission path. This is different from participant_surveys.py, which stores multi-choice answers as comma-separated OptionText strings; the two submission paths therefore produce differently formatted values in the same Responses table for the same question type. Any code reading from Responses must be able to handle both formats, and the aggregation service presumably does so, but this inconsistency is a maintenance liability worth resolving.

The Submission Endpoint:

The single endpoint runs a sequential validation pipeline before writing any data, rolling back on failure.

The first check confirms the survey exists and has PublicationStatus = 'published'. The second confirms the participant has a row in SurveyAssignment for this survey, returning a 403 if not. Notice that unlike participant_surveys.py, this endpoint does not check the assignment's current status; it does not verify that the assignment is pending rather than already completed or expired, and it does not check or enforce the due date. This means a participant could theoretically submit responses through this endpoint for an expired assignment or re-submit for a completed one, since neither condition is guarded against. The ON DUPLICATE KEY UPDATE clause present in participant_surveys.py is also absent here, so a duplicate submission would instead produce a MySQL integrity error that falls through to the bare except Exception block and returns a 500.

The third step builds a question_map dictionary from a single query that fetches all questions linked to this survey via QuestionList, mapping QuestionID to

ResponseType. This map is then used to validate each submitted response against the correct type for its question. For each response, if the question ID is not in the map, a 400 is raised immediately; a participant cannot submit answers to questions not in this survey. If the type validation fails, a 422 is raised with a descriptive error message. If both pass, the response is normalised and inserted.

The responses are inserted one at a time in a loop rather than in a batch, which means multiple round trips for surveys with many questions. If a validation failure occurs mid-loop, the transaction is not rolled back until the outer except Exception handler fires, which means some responses may have been inserted before the failure. Since the HTTPException re-raise path also does not call conn.rollback() explicitly, partial inserts from validation failures earlier in the loop would persist until the connection's implicit rollback on close, or they may be committed if the exception is somehow caught further up the stack. This is a subtle inconsistency; the except HTTPException: raise block should include a rollback to be safe.

After all responses are inserted, the assignment is marked as completed with CompletedAt = UTC_TIMESTAMP() and Status = 'completed'. The WHERE CompletedAt IS NULL clause on this update is a lightweight idempotency guard; if the assignment was somehow already completed, the UPDATE simply affects zero rows rather than overwriting the original completion timestamp.

Error handling here is more explicit than in most of the codebase. The bare except Exception block logs the error using Python's standard logging module before rolling back and raising a 500, which means unexpected failures will appear in the application log with a full traceback. This is better than the print(err) approach used in several other files and is a pattern worth adopting more broadly.

Sessions API:

This file manages the session lifecycle and provides the primary endpoint the frontend uses to understand the current user's identity and state. It also contains the core session infrastructure, token generation, hashing, and cookie configuration, that other parts of the codebase import and rely on.

Session Infrastructure:

The two constants at the top of the file establish the session behaviour: sessions expire after 8 hours (SESSION_EXPIRY_HOURS = 8), and a maximum of 5 concurrent active sessions are permitted per account (MAX_ACTIVE_SESSIONS = 5). Both are hardcoded rather than environment-configurable, so changing them requires a code deployment.

generate_session_token() produces a 32-byte URL-safe random token using Python's secrets module, resulting in a 43-character base64url string. hash_token() applies SHA-256 to the token before it is stored in the database. This means the Sessions table never contains the raw token, only its hash, so a database breach cannot be used to hijack active sessions directly. The raw token is set in the browser cookie and never stored anywhere in plaintext on the server side.

set_session_cookie() configures the cookie with three security attributes. httponly=True prevents JavaScript from reading the cookie value, providing protection against XSS attacks that attempt to steal session tokens. secure=True by default ensures the cookie is only sent over HTTPS connections; this can be overridden by setting the COOKIE_SECURE environment variable to "false" for local HTTP development, but this should never be disabled in any environment that handles real data. samesite="strict" prevents the cookie from being sent with cross-site requests, providing strong CSRF protection. Together these three attributes represent the current best practice for session cookie security.

Session Creation:

This function is not an endpoint, it is a shared utility imported and called by the auth module during login and MFA completion. It generates a token, inserts a new session row, enforces the concurrent session cap, fetches user metadata, and returns a dictionary containing the token alongside user details.

The concurrent session cap is enforced using an UPDATE statement that deactivates any active sessions beyond the most recent five, ordered by CreatedAt DESC. The MAX_ACTIVE_SESSIONS constant is interpolated directly into the LIMIT clause of a subquery, this is safe because it is an integer constant from application code rather than user input, but it is worth noting as a deviation from the parameterized-only approach used elsewhere. If a user has more than five active sessions (for example, after logging in on many devices without logging out), the oldest ones are revoked automatically on the next login.

After the session is created, the function fetches account details including role, consent version, birthdate, and gender, and uses these to compute two convenience flags: has_signed_consent (true if the user is an admin or has signed the current consent version) and needs_profile_completion (true if the user is a participant with missing birthdate or gender). These flags are returned in the session creation response so the frontend can immediately redirect to the appropriate screen after login without needing to make a separate /me call.

Importantly, the function returns the raw session token in the dictionary under "session_token", but the calling code in auth.py removes this key before including the dictionary in the HTTP response body. The token is delivered only via the cookie, never in the JSON response.

Session Validation:

This endpoint checks whether a token is currently valid and returns the associated AccountID. It accepts the token either from the request body (as data.token) or from the session cookie, body takes precedence. It does not check IsActive on the

session row, only that the row exists and that ExpiresAt is in the future. This means a session that was revoked via logout (which sets IsActive = 0 rather than deleting the row) would still pass validation here if it has not yet expired. Whether this is intentional or an oversight is worth verifying, if session revocation must be respected immediately, the query should include AND IsActive = 1.

Logout:

The logout endpoint reads the token exclusively from the cookie; unlike validation, it does not accept a token from the request body or Authorization header. This is a deliberate security choice: logout should only be triggerable by the party holding the cookie, not by anyone who happens to know the token string. It sets IsActive = 0 and RevokedAt = UTC_TIMESTAMP() on the matching session row and then clears the cookie from the browser. If no active session is found for the token hash, a 404 is returned. The soft-delete approach (marking inactive rather than deleting) preserves the session row in the database for audit purposes.

Session Info:

This is the most important endpoint in the file from the frontend's perspective. It is called on page load and after navigation to determine who is logged in and what state the session is in. It uses get_current_user as a dependency, which handles token extraction and validation before the handler runs, passing a user dict into the function.

The endpoint queries for the current user's account details, role, session expiry, ImpersonatedBy field, consent version, birthdate, and gender in a single join across AccountData, Roles, and Sessions. The ORDER BY s.ExpiresAt DESC LIMIT 1 clause means that if a user has multiple active sessions, the one expiring latest is used for the expiry timestamp; this is a reasonable heuristic but could theoretically show a session expiry different from the one the user is actually authenticated with.

The handler then branches through three distinct identity scenarios. The first and most common is a normal session: viewing_as_user_id is absent from the user dict, and ImpersonatedBy on the session row is null. In this case the user_info is built directly

from the account data and returned. The second scenario is the current view-as system: if user.get("viewing_as_user_id") is set, the handler fetches that user's details and constructs both a viewing_as object and an impersonation_info object, with the admin's own details pulled from the row already fetched. The user_info returned to the frontend in this case represents the viewed-as user rather than the admin, so the frontend can render the UI as if it were that user while still showing an impersonation banner. The third scenario is the legacy impersonation system: if ImpersonatedBy is non-null on the session row, a second query fetches the admin's details and populates impersonation_info. This branch exists to support sessions created by the deprecated /admin/users/{user_id}/impersonate endpoint and should become unreachable once that system is fully retired.

Consent status and profile completion are computed from the same row data as in create_session_for_account(), using the same logic: admins are always considered consented, and profile completion is only relevant for participants. These values being present on the /me response means the frontend can gate access to different screens on a single API call after initial load.

Surveys API:

This file handles the full lifecycle of surveys from creation through to closure. All endpoints require RoleID 2 (Researcher) or 4 (Admin) via a router-level dependency. Surveys exist in one of three publication states, draft, published, and closed, and the API enforces that transitions between these states follow a strict one-way path: draft to published via PATCH /{survey_id}/publish, and published to closed via PATCH /{survey_id}/close. There is no endpoint to revert a survey back to draft once published, nor to re-open a closed one.

The Survey Data Model:

The Survey table has two separate status fields, which is worth clarifying upfront since they appear together in most responses. PublicationStatus is the field actively managed by this API and drives access control throughout the platform; whether participants can be assigned to a survey, whether it can receive submissions, and so on. Status (mapped to a SurveyStatus enum of in-progress, complete, not-started, cancelled) appears to be a separate operational field that is returned in responses but never written to by any endpoint in this file. Its management presumably happens elsewhere or is set by the database directly, and maintainers should be aware that the two fields are independent.

Helper Functions:

Three helper functions are shared across multiple endpoints. get_survey_questions() fetches the full question list for a survey via a join between QuestionList and QuestionBank, including a secondary options query for single_choice and multi_choice questions, the same N+1 pattern noted in the question bank documentation. link_questions_to_survey() iterates a list of question IDs, verifies each exists in QuestionBank before inserting, and silently skips any that do not, invalid IDs produce no error, which means a caller who supplies a typo'd question ID will receive a survey with fewer questions than expected and no indication of what went wrong. If strict validation is needed here, the function should be updated to raise a 400 for

unrecognised IDs. unlink_all_questions() deletes all QuestionList rows for a survey and is used both during updates (to replace the question set) and during deletion (to clean up before removing the survey row).

A fourth helper, _fetch_survey_response(), consolidates the repeated pattern of fetching a survey row and building the SurveyResponse object. It is used by the publish and close endpoints to return the updated survey without duplicating the fetch logic.

Creating Surveys:

Surveys are created via POST /surveys, which accepts a title, optional description, optional publication status (defaulting to draft), optional start and end dates, and an optional list of question_ids to link immediately. The CreatorID is set from current_user["effective_account_id"], so surveys created while an admin is viewing-as a researcher will be attributed to the researcher rather than the admin. This is consistent with the view-as system's intent. After the survey row is committed, questions are linked if provided, translations are not triggered here (unlike question creation), and the newly created survey is re-fetched and returned.

Surveys can also be created from a template via POST /surveys/from-template/{template_id}. The endpoint fetches the template's title and description, then creates a new survey in draft status, optionally overriding the title and description from the request body. Questions are copied from TemplateQuestions to QuestionList in DisplayOrder sequence, preserving the template's intended question ordering. The new survey is always created as a draft regardless of what status any other survey created from this template might be in.

Listing and Fetching Surveys:

The list endpoint at GET /surveys supports optional filtering by publication_status and creator_id, with pagination via limit (1–1000, default 500) and offset. Unlike the single-survey fetch, the list response does not include the full question objects, it returns only question_count, computed with a separate COUNT(*) query per survey. This is again an N+1 pattern but a lighter one, since the count query is much cheaper than

fetching full question data for every survey in the list. The single-survey endpoint at GET /surveys/{survey_id} returns the full question list including options for choice questions.

Updating Surveys:

Updates to existing surveys are handled by PUT /surveys/{survey_id}, which builds its UPDATE statement dynamically from whichever fields are present in the request body, using the same conditional updates / params pattern seen throughout the codebase. A comment in the source acknowledges a known gap: there are currently no restrictions on editing a published survey's fields. The comment references "Task 009" as the planned location for adding status-based edit restrictions. Maintainers should be aware that as of this version of the code, a researcher can freely update the title, description, dates, and question list of a live survey that participants are actively completing. If question list updates are made to a published survey, participants mid-completion may encounter unexpected changes to the form.

If question_ids is supplied, the question update follows the same full-replace strategy as the question bank update, all existing links are deleted and the new set is inserted. Passing an empty list clears all questions from the survey without error, which could leave a published survey in a state where it has no questions. Combining this with the lack of edit restrictions on published surveys means this is a realistic risk in the current implementation.

Publishing and Closing Surveys:

Publishing a survey via PATCH /surveys/{survey_id}/publish enforces two pre-conditions before updating the status: the survey must currently be in draft state, and it must have at least one question linked. A survey with no questions cannot be published, attempting to do so returns a 400 with a descriptive message. Once published, it can receive assignments and submissions. The current status is included in the error message when the transition is invalid (for example, trying to publish an already-published survey returns the message "Current status is 'published', must be 'draft'"), which is useful for diagnosing state confusion on the frontend.

Closing a survey via PATCH /surveys/{survey_id}/close has only one pre-condition: the survey must currently be published. There is no validation that outstanding assignments have been completed before closing, so participants with pending assignments will find the survey closed when they try to submit. The submission endpoint in participant_surveys.py rejects submissions for non-published surveys with a 400, so this is a case where closing a survey mid-assignment would leave those participants unable to complete it. Whether this requires a warning or a grace period is a product decision, but it is worth documenting.

Deleting Surveys:

Deletion via DELETE /surveys/{survey_id} follows the same philosophy as question deletion: the relationships are cleaned up but the underlying data is preserved. Question links in QuestionList are removed, assignments in SurveyAssignment are deleted, and any Responses rows referencing this survey have their SurveyID set to NULL rather than being deleted, preserving participant response data even after the survey itself is gone. The survey row is then deleted. This means a deleted survey's response data remains in the Responses table as orphaned rows with a null SurveyID, and the aggregation service would need to handle this gracefully if it ever encounters such rows. There is no guard against deleting a published survey with active assignments, so deletion of a live survey is possible without warning.

Templates API:

This file provides CRUD management for survey templates; reusable survey skeletons that researchers can use as a starting point when creating new surveys. All endpoints require RoleID 2 (Researcher) or 4 (Admin) via the router-level dependency. Templates are structurally similar to surveys but simpler: they have no publication status, no assignment system, and no response data. Their primary purpose is to define an ordered question list that can be copied wholesale into a new survey via the from-template endpoint in surveys.py.

Templates vs Surveys:

The most important structural difference between templates and surveys is that templates store question order explicitly in TemplateQuestions.DisplayOrder, while surveys rely on the insertion order of QuestionList rows. This means get_template_questions() fetches questions with ORDER BY tq.DisplayOrder and includes the display_order value in the QuestionInTemplate response model, whereas the equivalent survey helper has no display order column and returns questions in whatever order the database returns them. When a survey is created from a template via surveys.py, it preserves this ordering by iterating TemplateQuestions sorted by DisplayOrder. Maintainers adding questions to either system should be aware of this difference in how ordering is managed.

Templates also have an is_public boolean field that surveys do not. A public template is visible to all researchers, while a private one is presumably intended only for the creator; however, the list endpoint does not enforce this visibility restriction. The is_public filter in GET /templates/ is an optional query parameter that the caller must supply explicitly; without it, all templates regardless of visibility are returned to any authenticated researcher. If creator-scoped template privacy is a requirement, the list endpoint would need a default filter based on the caller's identity.

Creating and Listing Templates:

Templates are created via POST /templates/, which accepts a title, optional description, an is_public flag (defaulting to false), and an optional list of question_ids. The CreatorID is set from current_user["effective_account_id"]. Question linking follows the same verify-before-insert pattern as surveys, with the addition that DisplayOrder is set to the question's index in the supplied list; so the order of question_ids in the request determines question order in the template. As with surveys, invalid question IDs are silently skipped rather than raising an error.

The list endpoint at GET /templates/ supports filtering by is_public and creator_id, with pagination via limit and offset. Like the survey list, it returns question_count rather than full question objects, with one count query issued per template in the result set. The single-template endpoint at GET /templates/{template_id} returns the full question list including options for choice questions.

Updating Templates:

Updates via PUT /templates/{template_id} follow the same dynamic field-building pattern as elsewhere. The is_public flag can be toggled, and the question list is replaced entirely if question_ids is supplied. One difference from the survey update is that the display order is re-assigned during a question list update; link_questions_to_template() uses enumerate() over the supplied list, so the order of IDs in the request body becomes the new display order. A partial question list update (keeping some existing questions while adding new ones) requires the caller to re-supply all desired question IDs in the correct order, since the existing links are deleted before the new ones are inserted.

Deleting Templates:

Deletion via DELETE /templates/{template_id} is simpler than survey deletion. The docstring notes that TemplateQuestions rows are removed via database-level CASCADE when the SurveyTemplate row is deleted, so no explicit cleanup of question links is needed in the handler. Questions themselves remain in QuestionBank.

Importantly, deleting a template has no effect on any surveys that were previously created from it; those surveys are independent copies that no longer reference the template once created. There is also no guard against deleting a template that researchers may be actively relying on, since there is no mechanism tracking which templates are in use

Duplicating Templates:

The duplicate endpoint at POST /templates/{template_id}/duplicate creates a new template that is an exact copy of the original's questions and description, with two deliberate modifications: the title gains a " (Copy)" suffix, and is_public is always set to false regardless of the original's visibility. The CreatorID is not set on the new template; the INSERT statement omits it, meaning the duplicate's creator will be null in the database. This is likely an oversight rather than intentional design, since creator attribution would be necessary for any future scoped visibility enforcement. The question links are copied with their original DisplayOrder values preserved, maintaining the question sequence from the source template. The endpoint returns the full new template including its questions.

TOS API:

This is the smallest file in the codebase, containing a single endpoint that records a user's acceptance of the platform's Terms of Service.

Duplicating Templates:

The endpoint is accessible to any authenticated user via get_current_user. It reads the current TOS version from the CURRENT_TOS_VERSION environment variable at module load time, captures the client's IP address using the same X-Forwarded-For / request.client.host fallback pattern as the consent API, converts it to packed binary with ip_to_varbinary(), and updates three columns on the caller's AccountData row: TosAcceptedAt (set to the current UTC time by the database), TosVersion, and TosAcceptedIp.

Several issues are worth flagging for maintenance. First, CURRENT_TOS_VERSION is read at import time with no default value, if the environment variable is not set, it will be None, and the database will store a null version string silently without any error. This is in contrast to the consent API, which provides a sensible default of "1.0". A startup assertion or at least a logged warning would make misconfiguration visible. Second, there is no error handling at all; no try/except around the database calls, no finally block to close the connection on failure, and no rollback if the UPDATE fails partway through. If a database error occurs, the connection will not be closed cleanly. Third, unlike the consent API, there is no separate audit record, acceptance is recorded only as a field update on AccountData, with no immutable history of when each version was accepted. This means if a user accepts a new TOS version, the record of their previous acceptance is overwritten. Fourth, there is no admin exemption equivalent to the one in the consent API, though whether admins should be subject to TOS acceptance at all is a product decision. Finally, the endpoint has no rate limiting, which is unlikely to be a problem in practice but is inconsistent with other user-facing write endpoints.

Two Factor Authentication API:

This file implements TOTP-based (Time-based One-Time Password) two-factor authentication. It covers three phases of the 2FA lifecycle, enrolling a new TOTP secret, confirming enrollment by verifying a code, and completing a login that was gated on a 2FA challenge, as well as a disable endpoint.

The file depends on a masterkey read from the 2FA_MASTER_KEY environment variable at import time. This key is used by encrypt_2fa_secret_for_db() and decrypt_2fa_secret_from_db() in core/totp.py to encrypt TOTP secrets before storage. If this environment variable is not set, masterkey will be None and every encryption or decryption call will fail at runtime. Unlike CURRENT_TOS_VERSION in the TOS file, this failure would surface immediately as a 500 error rather than silently storing a null, but it still warrants a startup assertion to make misconfiguration obvious during deployment.

Enrolling 2FA:

Enrollment is accessible to any authenticated user. The endpoint generates a fresh TOTP secret using pyotp.random_base32(), encrypts it with the master key, and upserts a row into Account2FA. The upsert uses ON DUPLICATE KEY UPDATE to handle the case where the user is re-enrolling after a previous setup; the existing secret is overwritten, IsEnabled is set back to 0, and VerifiedAt and LastUsedAt are nulled out, effectively invalidating the previous setup and requiring the user to confirm the new secret before 2FA becomes active again. This is the correct behaviour for a secret rotation flow.

The response contains only the provisioning_uri — a otpauth:// URI that encodes the secret, the user's email as the account name, and "HealthDataBank" as the issuer. The frontend is expected to render this as a QR code for the user to scan with their authenticator app. The raw secret is never returned to the client, which is the correct approach,the QR code encodes it but the API response itself does not expose it as a plain string.

Confirming Enrollment:

Confirmation is rate-limited to 5 attempts per 60 seconds per account. The endpoint fetches the encrypted TOTP secret for the caller's account, decrypts it, and verifies the submitted 6-digit code using pyotp.TOTP.verify() with valid_window=1. The valid_window=1 parameter means the verifier accepts codes from one 30-second window before and one window after the current time, accommodating minor clock drift between the server and the user's device. If the code is valid, IsEnabled is set to 1 and the timestamps are recorded.

The endpoint checks whether a row exists in Account2FA before attempting decryption, returning 404 if the user has not enrolled. However, it does not verify that the question being confirmed belongs to a recently initiated enrollment, a user who enrolled months ago, never confirmed, and then tries to confirm with a stale secret would reach the TOTP verification step. Since the secret would still be valid TOTP-wise (TOTP secrets don't expire), this is unlikely to cause practical problems, but it is worth noting.

Verifying a Login Challenge:

This is the most security-critical endpoint in the file. It completes the second step of login for users with 2FA enabled, consuming the challenge token issued by the login endpoint in auth.py and minting a real session cookie if the TOTP code is correct.

The challenge token is hashed with _hash_token() (imported from auth.py) and looked up in mfa_challenges. The endpoint checks four conditions before proceeding: the challenge must exist, it must not have been revoked, it must not have already been used (UsedAt is null), and it must not have expired. If any of these fail, a generic 401 is returned without distinguishing which condition triggered it, which is the correct approach for preventing information leakage about the challenge's state.

The attempt counter is incremented before the TOTP code is verified, this is intentional and noted in a comment. It means even a correct code counts against the attempt limit, which is a minor inconvenience but ensures the counter accurately reflects every

interaction with the challenge. If the attempt count has already reached MFA_MAX_ATTEMPTS (5), the challenge is revoked and a 429 is returned, preventing brute force against the 6-digit code space. A user who exceeds the limit must log in again to obtain a new challenge.

After a successful TOTP verification, the challenge is marked used via UsedAt = UTC_TIMESTAMP() and LastUsedAt on the Account2FA row is updated. The endpoint then calls create_session_for_account() to mint a full session, sets the session cookie, strips the raw token from the response body, and returns the session metadata. This is the same session creation flow as the non-MFA login path, so the resulting session is identical in every respect.

The rate limit here is applied per IP (5 per 60 seconds) rather than per account, which is a reasonable choice since the endpoint is accessible before a full session exists. However, an attacker who controls multiple IP addresses could still make many attempts across different IPs. The per-challenge attempt counter is the more meaningful protection here since it is tied to the specific challenge token rather than the source address.

Disabling 2FA:

The disable endpoint is accessible to any authenticated user and requires no additional verification beyond the session itself, a user does not need to supply their current 2FA code or password to disable 2FA. This is a notable security gap: if an attacker gains access to an active session (for example, through an XSS attack or a stolen cookie), they can immediately disable the account's 2FA protection without knowing the TOTP secret. Most platforms require the user to supply their current password or a valid TOTP code before allowing 2FA to be disabled. Adding such a verification step here would significantly reduce the risk of an attacker locking out the legitimate user's second factor.

The endpoint checks that Account2FA exists for the account and that it is currently enabled, returning descriptive 400 errors in both failure cases. If 2FA is enabled, it sets IsEnabled = 0. The TotpSecret is left in the database rather than being

cleared, meaning re-enabling 2FA by calling enroll again will overwrite it with a new secret, which is the correct

Users API:

This file provides two routers. The main router handles admin-facing user management operations and requires RoleID 4 (System Administrator) for all routes via a router-level dependency. A second self_router handles the single self-service endpoint that allows any authenticated user to update their own profile, with no role restriction beyond holding a valid session. Both routers are presumably mounted at different base paths in the application's router configuration.

Data Model and Helpers:

The UserResponse model includes fields that span three related tables: core account fields from AccountData, the role name joined from Roles, and consent metadata (ConsentSignedAt, ConsentVersion) also from AccountData. The LastLogin field is computed via a correlated subquery, SELECT MAX(s.CreatedAt) FROM Sessions WHERE s.AccountID = a.AccountID, rather than being stored directly. This means LastLogin reflects the most recent session creation time rather than the most recent actual login event, and on accounts with many sessions this subquery adds a per-row cost that accumulates in list queries.

The _as_utc() helper wraps naive datetime objects from MySQL with explicit UTC timezone information before they are serialised, which prevents ambiguity for any frontend or downstream system that cares about timezone-aware timestamps. The row_to_user_response() helper uses index-based access with len(row) guards, which makes it resilient to queries that return fewer columns, for example, the create endpoint's post-insert fetch passes NULL as LastLogin rather than the correlated subquery, and the helper handles this gracefully without needing a separate response-building path.

The fetch_user_by_id() function at the bottom of the file opens its own connection rather than accepting a cursor as a parameter. It is called from three endpoints, the two update endpoints and the status toggle, which means each of those endpoints opens one connection for the write operation and then fetch_user_by_id()

opens a second connection to re-fetch the result. This is functionally correct but less efficient than passing the existing cursor through, and is inconsistent with the _verify_hcp_access() pattern in hcp_patients.py which explicitly accepts a cursor to avoid this.

Listing and Fetching Users:

The list endpoint accessible at GET /users/ supports filtering by role name, active status, and a free-text search across first name, last name, and email using LIKE %search%. It runs two queries, a count query and a data query, with the same filter conditions applied to both, ensuring the total field in the response reflects the full filtered result set rather than just the current page. Pagination is via limit (1–500, default 100) and offset. Results are ordered by AccountID DESC, meaning the most recently created accounts appear first. The single-user fetch at GET /users/{user_id} uses the same query with a WHERE AccountID = %s clause, and returns 404 if no row is found.

Creating Users:

User creation via POST /users/ follows the two-step Auth-first pattern consistent with the rest of the codebase: an Auth row is inserted with the hashed password and MustChangePassword = TRUE, and then the AccountData row is inserted referencing the new AuthID. The role is resolved from the UserRole enum value to a RoleID by querying the Roles table, defaulting to RoleID 1 (Participant) if the lookup fails or no role is supplied.

Duplicate email detection happens at two levels. An explicit pre-check query runs before the insert, and a mysql.connector.IntegrityError catch handles the race condition where two concurrent requests might both pass the pre-check. This double-handling is good defensive practice. The response is built by re-fetching the created row with NULL as LastLogin in place of the correlated subquery, a minor optimisation since a newly created account will have no sessions anyway.

Updating Users:

The admin-facing update at PUT /users/{user_id} allows a system administrator to change any user's name, email, role, and active status. A comment in the source explicitly notes that this endpoint is believed to be unused by the frontend and is marked for removal in favour of the self-service endpoint below. If it is confirmed to be unused, removing it would reduce the attack surface, since it allows role escalation, an admin could promote any user to any role including System Administrator through this endpoint with no additional confirmation.

The self-service update at PUT /me (on self_router) allows any authenticated user to update their own first name, last name, email, birthdate, and gender. Unlike the admin update, it does not expose role or is_active, which is the appropriate scope for a self-service endpoint. Email uniqueness is checked before the update and handled both via an explicit pre-check and a mysql.connector.IntegrityError catch. The update is built dynamically from whichever fields are present in the request body, and the result is returned by calling fetch_user_by_id(), which opens a second database connection as noted above.

Status Toggle:

The status toggle at PATCH /users/{user_id}/status accepts is_active as a query parameter (a boolean) and updates IsActive on the AccountData row accordingly. It does not cascade, deactivating a user does not revoke their active sessions, so an already-logged-in user will remain authenticated until their session expires naturally. If immediate session invalidation is required upon deactivation, active sessions would need to be revoked here as well, which the account deletion request flow in auth.py does handle. This is worth noting as a gap if deactivation is intended to be an immediate access cutoff rather than a soft block on future logins.

Deleting Users:

The delete endpoint at DELETE /users/{user_id} is the most thorough cleanup operation in this file and has a self-deletion guard: if the calling admin's own account ID

matches the target, a 400 is returned immediately. The cleanup sequence is carefully ordered to handle foreign key constraints. Session references to the target user in ImpersonatedBy and ViewingAsUserID are NULLed first to avoid constraint failures when deleting the user's own sessions. Survey response data is anonymised by setting ParticipantID = NULL rather than deleted, preserving aggregate research data. Sent messages are deleted since SenderID is non-nullable. CreatorID on surveys and templates is NULLed so those records survive. ReviewedBy on account requests is NULLed for the same reason.

The AccountData row is then deleted. The docstring notes that several related tables, Account2FA, mfa_challenges, HcpPatientLink, ConversationParticipants, and FriendRequests, are cleaned up via ON DELETE CASCADE from the AccountData foreign key, rather than through explicit DELETE statements in the handler. This is a meaningful difference from the purge endpoint in admin.py, which handles some of these explicitly. Maintainers should verify that the cascade configuration in the database schema covers all of these tables as documented, since the correctness of this cleanup depends entirely on the schema's ON DELETE rules being correctly defined.

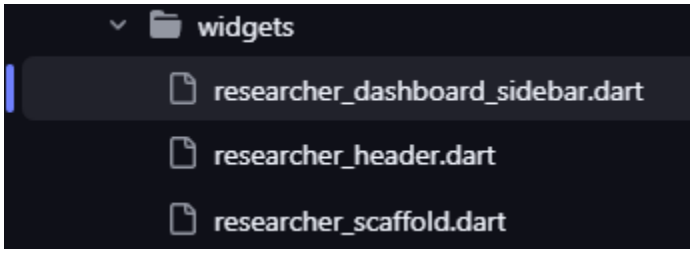
Flutter Widgets

The frontend of the application utilizes Flutter. Flutter utilizes Dart as its language, any member trying to modify the frontend should garner some familiarization with the language. Flutter allows for hot-reloading, allowing for changes to the UI without modifying the state of the application. It does this via injecting code into the Dart Virtual Machine, then rebuilds from the root so that all states are kept.

Flutter utilizes widgets as the fundamental building blocks of the UI. Everything within flutter is composed of widgets, which describe how the UI should look and behave based on the current state. Widgets are immutable. Instead, when the application changes, Flutter rebuilds the affected widgets.

Flutter widgets define both the structure and behaviour of the user interface. Complex UI components are built by composing smaller widgets into reusable structures. Stateless Widgets describe static layouts and configuration, while Stateful Widgets manage dynamic interactions like hover effects and visual state changes.

There are many flutter widgets within the application. Going over each would not be a good use of time, so this example will be utilizing the three Researcher widgets within the frontend/lib/src/features/researcher/widgets directory as an example, and the logic from here can be utilized on other widgets nearly one-to-one. Any questions about Dart the programming language can be answered utilizing the online Dart documentation.



Researcher Dashboard Sidebar:

This is the first of the widgets we will talk about. The ResearcherDashboardSidebar widget is a reusable navigation component that provides a scrollable sidebar for a dashboard. It is implemented as a stateless widget, meaning it does not manage its own internal state and instead relies entirely on the data passed through its constructor. The widget accepts a list of AppSectionNavbarSection objects, which define the hierarchical navigation structure along with an activeDestinationId to determine which item is currently selected. It also takes an asynchronous callback, onDestinationTap, which is triggered when a user selects a navigation item, enabling integration with scroll-based or route-based navigation logic.

Visually, the sidebar is constructed using a Container with a fixed width and styled using a gradient background, borders, and shadows defined in a centralized theme (AppTheme). Inside, a Column organizes the layout into three main parts: a header, a divider, and a scrollable navigation area. The header displays contextual user information (such as the logged-in user's name) and optionally includes a collapse button, allowing the sidebar to be programmatically hidden. Localization support is integrated through context.l10n, ensuring text such as labels and tooltips can adapt to different languages.

Navigation content is rendered dynamically by iterating over the provided sections. Each section is displayed using the private _SidebarSection widget, which renders a top-level item (the section heading) and its associated child items. This creates a two-level hierarchy where parent items can act as both labels and interactive destinations, while child items are visually indented to indicate their subordinate relationship.

Individual navigation items are implemented using the _SidebarItem stateful widget. This component manages hover state through a MouseRegion, enabling responsive visual feedback when the user's cursor enters or leaves the item. The widget conditionally applies styling based on three factors: whether the item is active, whether it is being hovered, and whether the application is using a "flat" or more

decorative theme style. These conditions influence properties such as background gradients, border styles, shadow effects, and text color. Active items are highlighted prominently, either through a gradient fill or a colored border, while hovered items receive subtle background changes to improve usability and affordance.

Interactivity is handled using an InkWell, which captures tap events and invokes the provided callback. The layout and spacing of each item are adjusted based on its role: section headers may use bold text, while child items are indented to reinforce the navigation hierarchy. Overall, the widget demonstrates a common Flutter design pattern: combining stateless structure with localized stateful components to create a highly customizable, theme-aware, and interactive user interface element.

Researcher Header:

The ResearcherHeader widget is a specialized implementation of a role-based navigation header, designed specifically for users in the “researcher” role. Rather than building a header from scratch, it extends the RoleBasedHeader base class, inheriting its layout, styling, and behavior while supplying researcher-specific configuration. This approach follows an object-oriented design pattern where shared functionality is centralized in a parent class, allowing role-specific headers to remain concise and consistent across the application.

The widget is constructed by passing key parameters such as the current route, notification handlers, and user information up to the parent class via super. The most important customization occurs in the initializer list, where two critical properties are defined: homeRoute and navItemsBuilder. The homeRoute establishes the default landing page for the researcher (in this case, the dashboard), while navItemsBuilder is a function that dynamically generates the list of navigation items displayed in the header.

The navItemsBuilder function takes the current build context and a value representing unread messages, and returns a list of NavItem objects. Each NavItem defines a label and route, which correspond to different sections of the researcher interface, such as surveys, templates, question bank, and data. Localization is handled through context.l10n, ensuring that all navigation labels adapt to the user’s language settings. Additionally, the messages navigation item includes a badge parameter, allowing it to display a notification count for unread messages, which enhances user awareness of pending activity.

By encapsulating all role-specific navigation logic within this subclass, the ResearcherHeader maintains a clean separation of concerns. The base RoleBasedHeader manages rendering and interaction behavior, while ResearcherHeader focuses solely on defining what content should be shown. This design improves maintainability, scalability, and consistency, especially in applications that support multiple user roles with distinct navigation requirements.

Researcher Scaffold:

The ResearcherScaffold widget is a high-level layout component that standardizes the structure and behavior of all researcher-facing pages in the application. It is implemented as a stateless widget that composes a pre-configured page layout by combining a shared BaseScaffold with a role-specific ResearcherHeader. This abstraction ensures consistency across pages while reducing duplication, allowing developers to quickly construct new screens with a uniform look and navigation experience.

At its core, ResearcherScaffold acts as a wrapper around BaseScaffold, passing through both required and optional configuration parameters. The only required inputs are currentRoute, which identifies the active navigation item in the header, and child, which represents the main content of the page. The ResearcherHeader is automatically injected into the scaffold's header property, ensuring that all researcher pages include the same navigation structure without requiring manual setup.

The widget exposes several optional parameters that control layout behavior and user interface features. These include scrollable, which determines whether the page content is wrapped in a scrollable container; showFooter, which toggles the presence of a footer; and floatingActionButton, which allows pages to include contextual actions. Layout customization is further supported through properties such as alignment, bodyBehavior, padding, and maxWidth, which integrate with the underlying page shell system to control spacing, responsiveness, and content positioning.

In addition to layout concerns, the scaffold also manages header-related interactions by forwarding properties such as hasNotifications, onNotificationsTap, onProfileTap, onMenuTap, and userName to the ResearcherHeader. This enables consistent handling of user actions like opening notifications or accessing profile settings across all pages.

Global Widgets:

Global widgets are reusable UI components that are designed to be used across multiple features or sections of an application, rather than being tied to a specific screen or role. They encapsulate common functionality or layout patterns, into centralized, consistent implementations. By defining these widgets in a shared location (e.g., a core or common module), developers can ensure uniform behavior and visual consistency throughout the application while reducing code duplication.

In practice, global widgets often serve as foundational building blocks for higher-level components. For example, a base scaffold or header widget can define standard layout structure, theming, and interaction patterns, while feature-specific widgets (such as researcher or admin views) extend or configure them. This separation of concerns improves maintainability, as updates to a global widget automatically propagate across all usages, and it enhances scalability by allowing new features to be built on top of well-defined, reusable components.

This application has many globalized widgets found in frontend/lib/core/widgets. With each file within the Widgets directory holding different types of widgets, labeled accordingly (the ‘buttons’ folder holding the buttons, for example). Button will be the one used to explain

The button widgets are a suite of five reusable Flutter components that standardize every tappable action across the HealthBank application. Rather than constructing Flutter’s raw ElevatedButton, TextButton, OutlinedButton, or custom InkWell containers directly on individual screens, all button UI flows through these wrappers. This means that visual decisions, colors, typography, padding, and disabled states, are made in one place and inherited everywhere automatically. The practical consequence is that the visual language of interactivity is consistent: a primary action looks the same whether it appears on a data entry form, a confirmation dialog, or an error recovery screen.

The entry point for consumers is the barrel file buttons.dart, which re-exports all five widgets under a single import. Any screen or widget that needs a button imports this one line and all five types become available immediately. Adding a new button variant to the folder and registering it in the barrel file makes it accessible across the entire codebase without touching any individual screen.

AppFilledButton

Defined in app_filled_button.dart, AppFilledButton is the primary action button of the application. It renders as a solid-background button using AppTheme.primary as the fill color by default, with AppTheme.textContrast (white) as the label color. This is the widget to reach for when a screen has one clear, most-important action: saving a form, confirming a decision, or submitting data.

Internally, AppFilledButton wraps Flutter's ElevatedButton. When onPressed is passed as null, Flutter automatically treats the button as disabled. The widget handles this gracefully by supplying AppTheme.muted as the disabledBackgroundColor and AppTheme.textContrast as the disabledForegroundColor, no conditional logic is required from the caller.

Padding and text size are responsive. On every build the widget reads the current screen width from MediaQuery, maps it to a breakpoint using breakpointForWidth(), and retrieves the corresponding bodyMedium text style from AppTheme.textThemeForBreakpoint(). Horizontal and vertical padding are derived proportionally from Breakpoints.responsivePadding(width). All of this happens automatically; callers that do not provide overrides get layout that is appropriate for the current device class. The widget exposes optional override parameters for cases where a non-standard appearance is needed: backgroundColor replaces the fill color, textColor replaces the label color, padding replaces the computed insets, and textStyle replaces the resolved typography.

AppLongButton

Defined in app_long_button.dart, AppLongButton is a full-width variant of AppFilledButton. Its implementation is intentionally minimal: it wraps AppFilledButton inside a SizedBox with width: double.infinity, causing the button to expand and fill whatever container it is placed in. It accepts exactly the same parameters as AppFilledButton and passes them straight through without adding any styling of its own.

Because AppLongButton is a thin wrapper with no independent style logic, any change to AppFilledButton is automatically reflected here. Use AppLongButton wherever a primary action should span the full available width, typically at the bottom of a form, inside a card, or within a dialog. Use AppFilledButton directly when the button should size to its label rather than stretching.

AppOutlinedButton

Defined in app_outlined_button.dart, AppOutlinedButton is a border-only button with no background fill. It wraps Flutter's OutlinedButton and is used for secondary actions that sit alongside a primary action, a Retry button next to an error state, a Cancel that accompanies a Confirm, or a less prominent action in a row of controls. Because it carries less visual weight than a filled button, it does not compete with the primary action for the user's attention.

The widget accepts an optional icon parameter. When provided, it switches internally to OutlinedButton.icon() and renders the icon to the left of the label at 18 logical pixels. When omitted, a plain OutlinedButton is rendered. The foreground color, applied to both text and icon, defaults to AppTheme.primary but can be overridden via foregroundColor. The border color defaults to the resolved foreground but can be set independently via borderColor, which is useful for destructive variants where both the border and label should read as AppTheme.error. Disabled state is handled automatically through Flutter's disabledForegroundColor, which the widget sets to AppTheme.muted.

AppTextButton

Defined in app_text_button.dart, AppTextButton is a label-only button with no border and no background. It wraps Flutter's TextButton and is intended for low-emphasis inline actions, "Learn more", "Forgot password?", "Skip", where a filled or outlined button would introduce unwanted visual weight and compete with surrounding content. The default text color is AppTheme.primary at FontWeight.w600. Like the other buttons, padding and text style are resolved responsively from the current breakpoint, and both can be overridden via textColor, textStyle, and padding. AppTextButton does not expose a disabled background color because there is no background to style; Flutter's default muting of the foreground color when onPressed is null applies automatically.

AppTabButton

Defined in app_tab_button.dart, AppTabButton is a toggle-style button used in filter rows and status selectors, situations where the user is choosing one option from a horizontal set of labeled choices. Unlike the other four widgets, AppTabButton does not extend a Flutter button primitive. It uses an InkWell wrapping a Container, which gives precise control over the fill, border, and corner radius in both selection states.

When isSelected is true, the container background becomes AppTheme.primary, the label switches to AppTheme.textContrast at FontWeight.w600, and the border matches the primary color. When isSelected is false, the background is colors.surface (read from context.appColors), the text is colors.textPrimary at normal weight, and the border uses colors.divider. This means AppTabButton's unselected appearance responds automatically to changes in the ambient color scheme without any modification to the widget itself.

AppTabButton is a purely controlled component, it holds no selection state internally. The parent widget is responsible for tracking which option is selected, passing the current value as isSelected, and calling setState inside the onTap callback to update it. The onTap parameter is required and non-nullable, since a tab with no action serves no purpose. No visual override parameters are exposed; the selected and unselected appearances are derived entirely from the application theme.

Relationship to AppTheme

All five button widgets read exclusively from AppTheme, a centralized theme definition that holds the application's color constants, text styles, and responsive layout helpers. None of the widgets own hard-coded colors or font sizes. Everything visual is resolved from the theme at render time, which means the button layer is a passive consumer of design decisions, not a place where those decisions are stored.

The color constants the buttons rely on are AppTheme.primary (used as the fill in AppFilledButton and AppLongButton, as the border and text in AppOutlinedButton, as the text in AppTextButton, and as the selected background in AppTabButton), AppTheme.textContrast (the label color on filled surfaces), AppTheme.muted (the disabled fill in AppFilledButton and the disabled foreground in AppOutlinedButton), and AppTheme.error (available for destructive variants of AppOutlinedButton via its override parameters). AppTabButton additionally reads colors.surface, colors.textPrimary, and colors.divider from the ambient context via context.appColors.

The responsive layout helpers used are breakpointForWidth(), AppTheme.textThemeForBreakpoint(), and Breakpoints.responsivePadding(). AppFilledButton, AppOutlinedButton, and AppTextButton call all three on every build. AppTabButton uses fixed padding constants, and AppLongButton inherits responsive layout from AppFilledButton rather than computing it independently.

Impact of Changes

Because these widgets are global, any modification propagates to every location where the widget is used. This is the intended design, but it means changes carry application-wide consequences that must be considered carefully before being made.

Modifying AppTheme directly, for example, changing the value of AppTheme.primary, is the highest-impact change possible. It will simultaneously alter the fill of every AppFilledButton and AppLongButton, the border and text color of every AppOutlinedButton, the text color of every AppTextButton, and the selected background of every AppTabButton across every screen in the application. This is the correct approach for a brand-wide color change, but it should be verified across all pages and interactive states before being committed.

Changing a default within a specific widget file, such as adjusting the padding multipliers in AppFilledButton, affects every instance of that widget but not the others, unless they delegate to it as AppLongButton does. Any such change should be tested across all breakpoints and alongside all widgets that appear in the same layout, since padding changes can cause misalignment with adjacent elements.

Adding a new optional parameter to any widget is a non-breaking change, provided the parameter is nullable and its absence produces the existing default behaviour. Existing call sites compile and render identically without modification. Removing a parameter or changing its type is a breaking change: every call site that passes that parameter will fail to compile. Changing a parameter’s default value is semantically breaking even if it compiles cleanly, since it silently alters the appearance of every call site that relied on the former default.

How to Extend or Modify

The most common task is changing the default visual style globally. The correct place to make this change is AppTheme, not the button files themselves. Updating a color constant or typography definition there propagates automatically into all five buttons without requiring any modification to the widget layer.

To add an optional feature to an existing widget, for example, a trailing icon on AppFilledButton, add a nullable IconData? parameter to the constructor, store it as a final field, and handle it conditionally in the build method. The child of the ElevatedButton can be replaced with a Row containing the label Text and the Icon when the parameter is non-null, and left as a plain Text when it is null. Because the parameter is optional with a null default, all existing call sites continue to compile and render without change.

To add an entirely new button variant, create a new file in the buttons/ directory using the naming convention app_<type>_button.dart, and add the corresponding export line to buttons.dart. The new widget becomes available to all consumers of the barrel import immediately. Follow the same constructor pattern as the existing widgets: required parameters for the essential inputs such as label and onPressed, optional nullable parameters for any style overrides, and a super.key pass-through.

AppTabButton is the one widget that does not follow the responsive padding pattern of the others. If the application's breakpoint system is later extended to cover additional device classes and tab buttons should adapt accordingly, the fixed EdgeInsets.symmetric(horizontal: 16, vertical: 8) inside AppTabButton should be replaced with the same Breakpoints.responsivePadding() call used by the other four widgets.

Audit Log

The audit middleware is a Starlette BaseHTTPMiddleware subclass that intercepts every HTTP request passing through the application and writes a corresponding record to the AuditEvent database table. Its purpose is to produce a complete, tamper-evident log of all system activity, who did what, from where, and with what outcome, without requiring individual route handlers to implement any logging logic themselves. Because it operates at the middleware layer, it runs unconditionally for all requests regardless of which endpoint is called or which user role initiated the action.

How It Works

When a request arrives, the middleware records its start time and assigns it a unique request ID, either by reading the x-request-id header if one was provided by a upstream proxy, or by generating a fresh UUID. This ID is attached to request.state so that downstream handlers can reference it, and it is echoed back in the response headers. If the request path appears in the configured exclude_paths set, the middleware short-circuits immediately and returns the response without writing any audit record. This mechanism exists to suppress logging for paths where recording would be counterproductive, such as internal health checks that would otherwise flood the audit log.

For all other requests, the middleware collects the client IP address, user agent string, HTTP method, path, and query string before passing the request to the next handler in the chain. The client IP is resolved from the x-forwarded-for header if present, taking only the first address in the chain, which represents the original client rather than any intermediate proxies. If that header is absent, it falls back to the direct connection address from request.client.host. The IP is then converted to its packed binary representation for storage, which supports both IPv4 and IPv6 addresses in a fixed-width column without requiring separate handling per address family.

The request is then dispatched to the actual route handler via call_next. The middleware wraps this in a try/except/finally block. If the handler raises an uncaught

exception, the exception class name is captured and the exception is re-raised immediately so that FastAPI's normal exception handling continues uninterrupted. Regardless of whether the handler succeeded, raised, or returned an error response, the finally block always executes and performs the audit write. This guarantees that even failed or crashing requests are recorded.

Semantic Action Classification

Rather than logging raw HTTP method and path strings as the action description, the middleware maps each request to a human-readable semantic action name and a resource type. This mapping is defined in the _ROUTE_ACTIONS list at the top of the file, an ordered list of tuples, each specifying an optional HTTP method, a compiled regular expression pattern, an action name, and a resource type. The _classify_request function iterates this list in order and returns the action and resource type of the first matching entry. If no entry matches, it falls back to the generic action "http_request" with resource type "HTTP".

The ordering of entries in _ROUTE_ACTIONS is significant. More specific patterns must appear before more general ones that could match the same path. For example, GET /api/v1/surveys/{id}/assignments must be listed before GET /api/v1/surveys/{id}, because the latter pattern would otherwise match first and produce an incorrect action name. Entries where the method field is None match any HTTP method, which is used for endpoints like consent operations where the action is meaningful regardless of verb.

Individual route handlers can override the classification determined by _ROUTE_ACTIONS by setting request.state.audit_action and request.state.audit_resource_type before the response is returned. The middleware checks for these attributes in the finally block and uses them in place of the automatically classified values if present. This is the correct mechanism for handlers that perform context-dependent actions whose meaning cannot be determined from the URL pattern alone.

What Gets Written

Each audit record contains the request ID, actor account ID and actor type (both read from request.state, where they are expected to be set by the authentication middleware that runs earlier in the stack), the binary-encoded IP address, the user agent string, the HTTP method, the path, the classified action name and resource type, a composite resource ID formed from the method and path, the semantic status string, the raw HTTP status code, an error code if an unhandled exception occurred, and a JSON metadata blob containing the request duration in milliseconds, the query string, and the content-length byte count.

The semantic status is derived from the HTTP status code by _status_from_http: codes 401 and 403 produce "denied", codes in the 200–399 range produce "success", and everything else produces "failure". Unhandled exceptions that cause the handler to crash before returning any response are recorded with an HTTP status code of 500 and an ErrorCode of "UNHANDLED_EXCEPTION".

The database write itself is performed inside a nested try/except block. If the write fails for any reason, a dropped connection, a schema mismatch, a database timeout, the error is logged at the ERROR level but the exception is swallowed. This is intentional: the audit system must never be the cause of a user-facing request failure. The comment in the code states this explicitly. The consequence is that audit records can be silently lost under database failure conditions, which should be accounted for in any compliance requirements that treat the audit log as authoritative.

Modifying the Route Map

To add a new route to the semantic classification, append a tuple to _ROUTE_ACTIONS in the appropriate section. The tuple format is (method_or_None, regex_pattern, action_name, resource_type). The pattern is matched against the path only, not the full URL, and is anchored at the start with ^. If the new route could be matched by an existing more-general pattern, the new entry must be placed above that pattern in the list.

To exclude a path from audit logging entirely, pass it in the exclude_paths set when registering the middleware. To add a context-sensitive action from within a route handler, assign to request.state.audit_action and optionally request.state.audit_resource_type at any point before the handler returns. The middleware will pick up these values in its finally block.

Security Structure

This is just a small section to show what different types of security features the app has. We recommend that administrators or technicians use this as a base reference for features and not a direct explanation on the exact way that each of these processes work; too in-depth of an explanation here may result in security concerns regarding this document. Most files in the repository have developer documentation in the docs folder, it's recommended to seek those out to learn more strict functionality. There is also inline documentation for many of these features.

Password

Upon creation of an account, all users will be forced to make a password. Passwords must fit a simple criteria to ensure they're not too simple, then will be stored securely within the database after going through a Salt and Hash algorithm to obfuscate them.

Two Factor Authentication

The application supports Two Factor Authentication, and all members have it turned on by default. This can be disabled in the settings option, but is not recommended. The Two Factor Authentication can either be done through a QR code reader such as Microsoft Authenticator or via email.

Sanitization

Sanitization, via the backend/app/utils/sanitize.py file, does a simple task. It takes all inputs and cleans them of any potentially malicious code or executables before being allowed to enter the database.

Session Tokens

The application utilizes HTTP only cookies for web, and authorization bearer tokens for mobile. They're a simple layer of security that allows for tracking API requests and ensuring that users are who they say they are.

Salt/Hash

All relevant data is set through a Salt and Hash algorithm. A standardized amount of data is injected (salting), and then it is pushed through a mathematical function to turn it into a fixed-length string of characters. This is how passwords are stored server-side, meaning the actual passwords of the users aren't on the database.

Anonymization

All user information is anonymized by removing any indicators of personal information. Upon export, the individual user is represented by a 'user id' that is a mix of numbers and letters. These numbers and letters are randomly created on instantiation of the user account.

Backup

The database within the application will automatically create backups after a certain period has passed (defined by administrators) or when one is forced. These backups can be used to refill the database should the living application be damaged, though some data will inevitably be lost; it is recommended that backups are taken regularly.

Rate Limiting

Users only have a limited amount of API calls per minute before the session is forcefully terminated. This prevents brute force attacks or malicious probing, and can mitigate Denial-Of-Service attacks.

Audit Log

The audit log holds information about people who make requests of the website such as the IP address. Utilizing the search or filter function within it can make tracking down potentially vulnerable accounts significantly easier, and said accounts can be deleted or deactivated depending on need.

Recommendations

It is recommended that, upon expansion of the application, HTTPS be implemented. It is a minor security vulnerability for now, and due to time constraints could not be implemented. While it doesn’t pose any immediate threats, it could cause small bits of personal data to be leaked should an individual log into a public wifi network.

For a minor recommendation, the audit log could be updated to flag potentially dangerous requests for review by Administrators. This could inform Administrators of accounts that may have been broken into or damaged before any actual damage can be committed.